Exploratory Data Analysis

EDA & Data Preprocessing

A complete practical guide, worked end to end on the Titanic dataset.

by Bhuvnesh Verma

github.com/MasterBhuvnesh/notebooks

Contents

1 Introduction to EDA **2**
1.1 What is EDA? . . . 2
1.2 Why EDA matters before machine learning . . . 2
1.3 EDA versus data preprocessing . . . 3
1.4 The data science workflow . . . 4
1.5 The running example: the Titanic dataset . . . 5

2 Loading the Data **7**
2.1 The four libraries . . . 7
2.2 The six functions you run first . . . 7
2.3 Reading the output like an analyst . . . 8

3 Understanding Data Types **10**
3.1 The two families and the four types . . . 10
3.2 Why the classification drives everything downstream . . . 11
3.3 Titanic, column by column . . . 11
3.4 Code . . . 12

4 Missing Value Analysis **14**
4.1 Why missing values are dangerous . . . 14
4.2 The three mechanisms . . . 14
4.3 Finding them . . . 14
4.4 The decision guide . . . 15
4.5 Mean, median and mode imputation . . . 17
4.6 Skewness: the number that decides mean versus median . . . 18
4.7 Titanic: the three decisions, with numbers . . . 20
4.8 Code, from naive to production . . . 20
4.9 Beyond mean, median and mode . . . 21

5 Duplicate Data **24**
5.1 Why duplicates harm a model . . . 24
5.2 Detection . . . 24
5.3 Removal . . . 24
5.4 When NOT to remove duplicates . . . 25

6 Outlier Analysis **27**
6.1 Why outliers matter . . . 27
6.2 Valid outlier or data error? . . . 27
6.3 The boxplot . . . 29
6.4 The IQR method . . . 29
6.5 Other detection methods . . . 30
6.6 Remove, cap or leave . . . 31
6.7 Winsorization, capping and transformation . . . 31

7 Univariate Analysis **34**
7.1 Numerical variables . . . 35
7.2 Categorical variables . . . 36

8 Bivariate Analysis **38**
8.1 Numerical versus numerical: the scatter plot . . . 38
8.2 Categorical versus numerical: the box plot . . . 38
8.3 Categorical versus the target . . . 39

9 Multivariate Analysis **41**
9.1 The pair plot . . . 41
9.2 The correlation matrix and heatmap . . . 41
9.3 The four correlation traps . . . 42
9.4 Three variables at once . . . 43

10 Feature Engineering **45**
10.1 FamilySize and IsAlone . . . 46
10.2 Title extraction from Name . . . 46
10.3 Everything else worth building here . . . 47
10.4 Feature engineering can leak . . . 48

11 Encoding Categorical Variables **50**
11.1 Label encoding . . . 50
11.2 One-hot encoding . . . 51
11.3 The Titanic decisions . . . 52
11.4 The other encoders, and when they are worth it . . . 53

12 Feature Scaling **55**
12.1 Which algorithms need it, measured . . . 55
12.2 Standardization (Z-score) . . . 56
12.3 Normalization (Min-Max scaling) . . . 56
12.4 Standardization versus normalization . . . 57

13 Train / Test Split **61**
13.1 Data leakage . . . 61
13.2 Train, validation and test . . . 62
13.3 The code, and the three arguments . . . 62
13.4 Cross-validation . . . 63

14 Complete Titanic Pipeline **66**
14.1 Steps 1 and 2: load and understand . . . 66
14.2 Steps 3 to 5: cleaning decisions, recorded . . . 67
14.3 Step 7: feature engineering, row-wise and leak-free . . . 67
14.4 Step 8: split, before anything is fitted . . . 68
14.5 Step 9: one transformer for the whole matrix . . . 68
14.6 Steps 10 to 12: train, evaluate, cross-validate . . . 68
14.7 The results . . . 69
14.8 Production checklist . . . 70

15 Industry Best Practices **71**
15.1 What changes when the work goes to production . . . 71
15.2 Common beginner mistakes, collected . . . 72
15.3 Checklist: before training any model . . . 73
15.4 Checklist: before deploying any model . . . 73

Appendix: The Guide on One Page **74**

1. Introduction to EDA

1.1 What is EDA?

Short description: Exploratory Data Analysis is the practice of interrogating a dataset before modelling it: summarising it numerically, drawing it, and forming hypotheses about what generated it. The term is John Tukey's, from his 1977 book of the same name, and his framing still holds: EDA is detective work, not confirmation.

The distinction that matters is between *confirmatory* analysis, where you arrive with a hypothesis and test it, and *exploratory* analysis, where you arrive with nothing and let the data suggest the hypotheses. EDA is the second. You are not trying to prove anything yet. You are trying to find out what you are holding.

Concretely, EDA answers:

- How large is the data, and what does one row represent?
- What type is each column really, as opposed to what type pandas guessed?
- What is missing, how much, and is the missingness itself a signal?
- How is each variable distributed, and is it skewed?
- Which values are impossible, and which are merely unusual?
- Which variables move together, and which move with the target?
- Which features look predictive enough to be worth engineering?

The output of EDA is not a chart. It is a list of decisions: drop this column, impute that one with the median, cap this variable, build that feature. The charts are how you reach the decisions; the decisions are the deliverable.

1.2 Why EDA matters before machine learning

A model is a function that takes your data at face value. It has no way to notice that a fare of 512 pounds belongs to three people sharing one ticket, that `Cabin` is missing for 77% of passengers, or that you accidentally left the passenger ID in the feature matrix. Every one of those problems is invisible in the accuracy score until it is expensive.

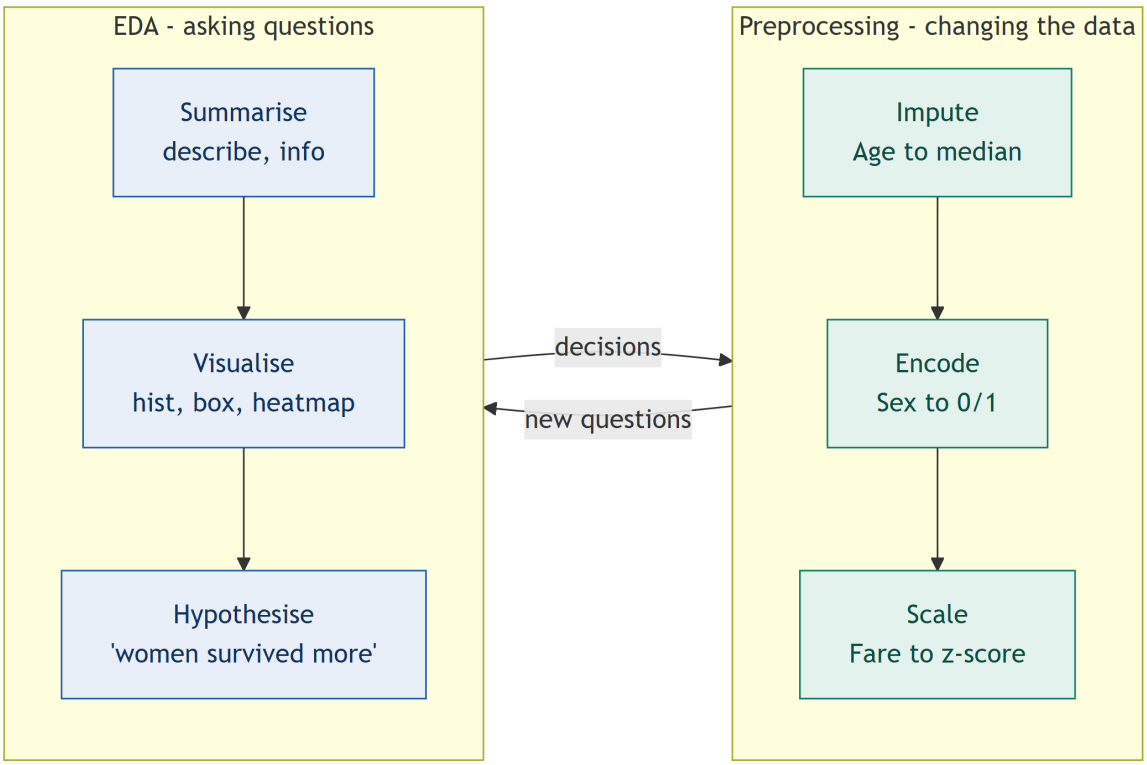
Skipping EDA costs you	What it looks like in practice
Silent data errors	An age of 250 or a negative fare trains the model as if it were real.
Wrong imputation	Filling a skewed variable with its mean drags every imputed row toward the tail.
Leaked features	A column that encodes the answer gives 99% accuracy in testing and random results in production.
Wasted modelling time	Weeks tuning hyperparameters on a feature set that one new feature would have beaten.
Wrong metric	99% accuracy on data that is 99% one class, which a constant predictor also achieves.
Undetectable failure	The model works, then quietly degrades, and nobody can say what normal looked like.

The economic argument: Practitioner surveys have long put data understanding and preparation at roughly 60–80% of the time spent on a typical project. Whether the real figure in your team is 50% or 90%, the ranking is not in dispute: this is the largest single block of work in applied machine learning, and it is the block where mistakes are cheapest to fix and most expensive to miss.

1.3 EDA versus data preprocessing

Beginners run the two together, and for good reason: in a notebook they interleave. They are still different activities with different outputs, and confusing them is how people end up scaling their test set with statistics learned from the whole dataset.

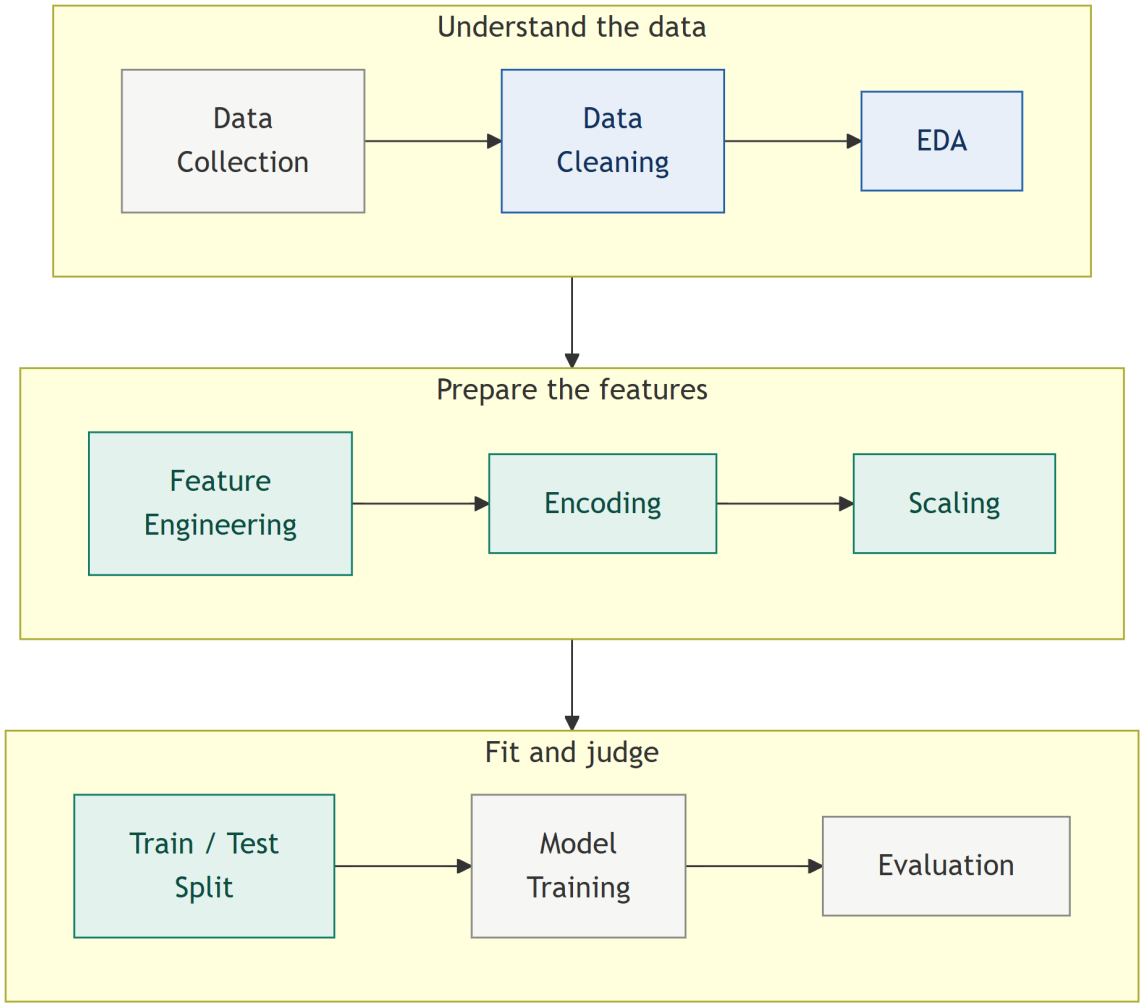
	EDA	**Preprocessing**
Purpose	Understand the data	Make the data usable by a model
Question	What is going on here?	What has to change?
Changes the data	No	Yes
Output	Insight, decisions, charts	A transformed matrix
Reversible	Trivially, you changed nothing	Not always
Runs on	The training data	Fitted on train, applied to test
Typical tools	`describe`, `value_counts`, seaborn	`SimpleImputer`, `OneHotEncoder`, `StandardScaler`
Titanic instance	`Age` is missing for 177 rows and skews right	Fill `Age` with the median of its title and class group



The loop is the point: EDA produces a preprocessing decision, the preprocessing changes the data, and the changed data deserves another look. Imputing `Age` with a single median puts 177 identical values into the middle of the distribution, which is visible immediately in a histogram

and invisible in the mean.

1.4 The data science workflow



Stage	What happens, and why it is here
Data Collection	Pull from the CSV, database, API or warehouse. Record where it came from and when; you will need that when the numbers change.
Data Cleaning	Fix types, impossible values, duplicates and obvious corruption. Cheap structural repairs, before you form any opinion.
EDA	Summarise, plot, correlate. Produce the decision list.
Feature Engineering	Build the columns the model cannot derive on its own: `FamilySize`, `Title`, `IsAlone`.
Encoding	Turn categories into numbers a model can consume.
Scaling	Put numeric features on a comparable range, for the algorithms that care.
Train / Test Split	Hold back data the model never sees, so evaluation means something.
Model Training	Fit the estimator on the training data only.
Evaluation	Score on the held-out set with a metric that matches the problem.

Where EDA fits: Third, after the structural cleaning and before every decision that follows. It is also the only stage you return to. Evaluation telling you the model confuses third-class women with third-class men sends you back to look at the data again, not to the hyperparameter grid.

An important correction to that diagram: the ordering above is the conventional teaching sequence, and it contains a trap. Splitting *after* scaling means the scaler learned its mean and standard deviation from rows that are supposed to be unseen. In real code the split moves earlier, immediately after feature engineering, and everything that *learns a parameter from the data* (imputer, encoder, scaler) is fitted on the training fold alone. Section 13 covers this properly; the corrected pipeline is the one in Section 14.

1.5 The running example: the Titanic dataset

Every technique in this guide is demonstrated on the same 891-row Titanic passenger manifest, the standard training split used in the Kaggle competition. It is small enough to read and awkward enough to be instructive: it has a skewed monetary variable, a column that is 77% missing, a free-text column hiding a useful feature, and a target imbalanced just enough to matter.

Column	Type	Meaning
`PassengerId`	Identifier	Row number, 1–891. Never a feature.
`Survived`	Target, binary	0 = died, 1 = survived.
`Pclass`	Ordinal	Ticket class: 1 = upper, 2 = middle, 3 = lower.
`Name`	Text	Full name, including a title such as *Mr* or *Master*.
`Sex`	Nominal	`male` or `female`.
`Age`	Continuous	Years; fractional under 1. Missing for 177 rows.
`SibSp`	Discrete	Siblings and spouses aboard.
`Parch`	Discrete	Parents and children aboard.
`Ticket`	Text	Ticket number, shared by travelling groups.
`Fare`	Continuous	Price paid, in pounds, for the whole ticket.
`Cabin`	Text	Cabin number. Missing for 687 rows.
`Embarked`	Nominal	Port: C = Cherbourg, Q = Queenstown, S = Southampton.

The headline numbers, computed on the file: 891 rows, 12 columns; 342 passengers survived and 549 did not, a survival rate of 38.38%; `Age` is missing 177 times (19.87%), `Cabin` 687 times (77.10%) and `Embarked` twice (0.22%); there are no duplicated rows.

Common mistakes

- **Treating EDA as a chart-generation ritual.** Twenty plots and no written conclusions is not EDA; it is procrastination with a colour palette.
- **Exploring the test set.** Every summary you compute on held-out data leaks into your decisions, even when no code touches it. Explore the training split.
- **Starting with the model.** Fitting a random forest to see what happens wastes the one part of the pipeline where you still had cheap options.
- **Not writing anything down.** An insight you cannot restate in one sentence will not survive to the next working session.

Industry best practices

- Keep a running **decision log**: one line per finding, one line per action taken. It becomes the model card and the handover document.
- Fix the **random seed** in the first cell and never change it mid-analysis.
- Treat the notebook as a **draft**, and port every decision into a script or `sklearn` pipeline. A notebook that ran once and cannot be replayed is not a result.
- Record the data's **provenance and version**. The CSV on the shared drive is not a source.

Interview questions

1. **What is EDA and why is it necessary?**
   Systematic inspection of a dataset before modelling, to establish structure, quality, distribution and relationships. Necessary because every downstream choice depends on facts only EDA supplies.
2. **Difference between EDA and data preprocessing?**
   EDA observes and does not modify; preprocessing modifies. EDA produces decisions, preprocessing executes them.
3. **Where does EDA sit in the pipeline, and is it a one-off?**
   After cleaning, before feature engineering, and it is a loop: evaluation results send you back into it.
4. **Should EDA be done on the full dataset or only the training set?**
   The training set. Decisions made after looking at the test set are a subtle form of leakage that inflates your reported score.
5. **How would you explore a dataset with 5000 columns?**
   Not column by column. Automate the profile, missingness, cardinality, dtype, variance, rank by usefulness, then inspect the top candidates and the target relationship manually.

2. Loading the Data

2.1 The four libraries

Library	Role	What you use it for here
`pandas`	Tabular data	`DataFrame` and `Series`: loading, indexing, grouping, joining, missing-value handling. The workhorse.
`numpy`	Numeric arrays	The array type underneath pandas. Vectorised maths, `np.nan`, `np.log1p`, random numbers.
`matplotlib`	Plotting engine	Figures, axes, labels, saving. Everything seaborn draws is a matplotlib object underneath.
`seaborn`	Statistical plots	One-line histograms, boxplots, countplots and heatmaps with sensible defaults.

Why both matplotlib and seaborn: seaborn gives you the plot in one call; matplotlib gives you control over the result. The normal pattern is a seaborn call for the chart and matplotlib calls for the title, the axis labels and the figure size.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.max_columns", None)   # do not hide columns behind "..."
pd.set_option("display.width", 200)
sns.set_theme(style="whitegrid")             # readable defaults for every plot
RANDOM_STATE = 42                            # one seed, defined once

df = pd.read_csv("Titanic.csv")
print(df.shape)                              # (891, 12)
```

2.2 The six functions you run first

Call	What it tells you
`df.shape`	Rows and columns as a tuple. The first sanity check: 891 rows and 12 columns is what the Titanic training file should have.
`df.head()`	First five rows. Confirms the header parsed, the delimiter was right, and one row means one passenger.
`df.tail()`	Last five rows. Catches trailing junk: a totals row, a stray footer, a block of empty lines.
`df.info()`	Per column: non-null count and dtype, plus memory use. The fastest way to see both missingness and a wrongly inferred type.
`df.describe()`	Count, mean, std, min, quartiles and max for numeric columns. Where you first see skew and impossible values.
`df.columns`	The column names as an Index. Catches trailing spaces and inconsistent casing before they cause a `KeyError`.

2.3 Reading the output like an analyst

```
df.head()
df.tail()
df.info()
df.describe()
df.describe(include="object")     # categorical columns: count, unique, top, freq
df.columns.tolist()
df.dtypes
```

Running `df.describe()` on the Titanic file gives the table below. Four facts jump out of it before any plot is drawn.

	count	mean	std	min	25%	50%	75%	max
Survived	891	0.38	0.49	0	0	0	1	1
Pclass	891	2.31	0.84	1	2	3	3	3
Age	714	29.70	14.53	0.42	20.12	28.00	38.00	80.00
SibSp	891	0.52	1.10	0	0	0	1	8
Parch	891	0.38	0.81	0	0	0	0	6
Fare	891	32.20	49.69	0.00	7.91	14.45	31.00	512.33

1. **`Age` has count 714, not 891.** `describe` silently skips nulls, so a count below the row total is a missing-value report. 177 are missing.
2. **`Fare` has a mean of 32.20 and a median of 14.45.** Mean far above median means a long right tail. This single comparison decides that `Fare` gets a median, never a mean.
3. **`Fare` runs from 0 to 512.33 with a standard deviation of 49.69.** The maximum is nearly ten standard deviations from the mean. Something is going on at the top of that column.
4. **`Pclass` and `Survived` are in the table at all.** They are categorical, and a mean `Pclass` of 2.31 is meaningless. `describe` summarises whatever is numeric, not whatever is quantitative; the difference is yours to enforce.

The categorical half: `df.describe(include="object")` is the call people forget. It reports `Name` with 891 unique values out of 891 (an identifier, not a feature, but it hides one), `Ticket` with 681 unique out of 891 (so 210 passengers share a ticket with somebody, which will matter for `Fare`), and `Cabin` with 147 unique across only 204 non-null rows.

Common mistakes

- **Reading `describe` and moving on.** The count row is a missing-value report and the mean-median gap is a skew report. Both are free and both get skipped.
- **Trusting inferred dtypes.** One stray string turns a numeric column into `object`, and every arithmetic operation on it silently changes meaning.
- **Not checking `df.tail()`.** Export tools append summary rows. A totals row read as a passenger is a data error you introduced at load time.
- **Using `inplace=True` everywhere.** It saves a variable name and costs you the ability to rerun the cell. Reassign instead.

Industry best practices

- Load with **explicit dtypes** once you know them, by passing a `dtype` mapping to `pd.read_csv`. It documents intent and prevents silent re-inference.
- Keep the **raw dataframe untouched** and work on a copy: `raw = pd.read_csv(...)` then `df = raw.copy()`. When a transformation goes wrong you reload from memory, not from disk.
- Assert the shape after loading. `assert df.shape == (891, 12)` turns a silent parsing change into a loud failure.
- For large files, use `nrows` while developing and remove it for the real run.

Interview questions

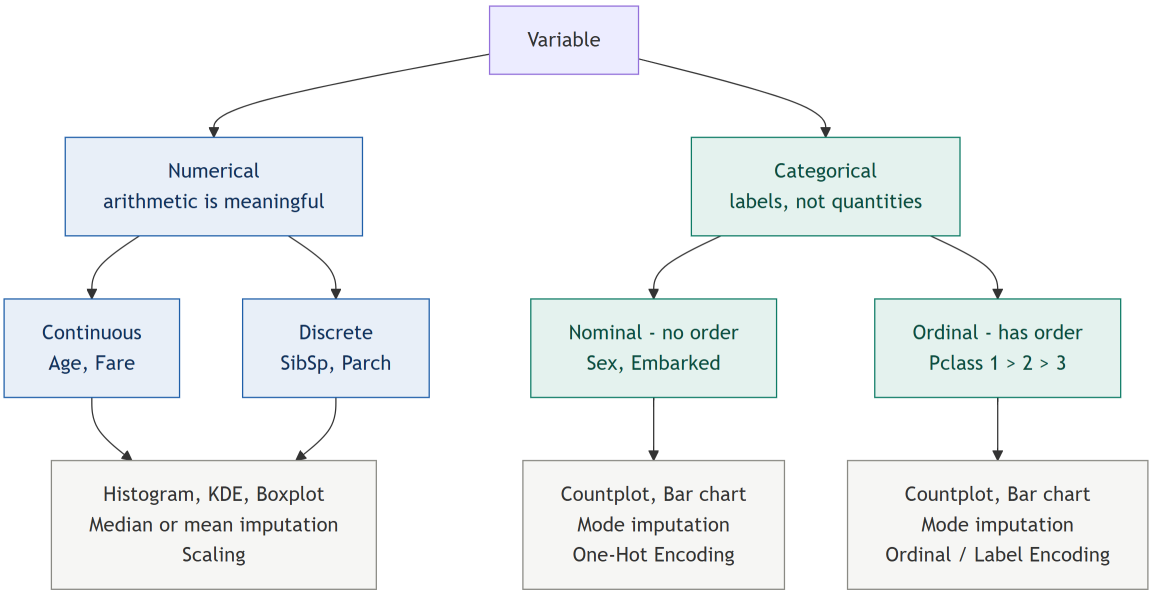
1. **What does `df.info()` show that `df.describe()` does not?**
   Dtypes, non-null counts for every column including object ones, and memory usage. `describe` covers numeric columns only unless told otherwise.
2. **Why can `count` differ between columns in `describe`?**
   It counts non-null values, so a lower count is missing data.
3. **Mean 32.20, median 14.45. What do you conclude?**
   Strong right skew. Use the median for imputation and consider a log transform or a robust scaler.
4. **How do you summarise categorical columns?**
   `df.describe(include="object")` for count, unique, top and freq, and `value_counts(dropna=False)` per column.

3. Understanding Data Types

Short description: Before any chart or transformation, every column has to be classified. The classification is not the pandas dtype. It is what the values *mean*, and it determines the plot you draw, the statistic you compute, the imputation you choose and the encoder you apply.

Why it is needed: Every downstream tool asks the same question in different words. A histogram needs a continuous variable. A countplot needs a categorical one. A mean needs a quantity. One-hot encoding needs an unordered category. Get the classification wrong and the tool still runs; it just answers a question you did not ask. A mean `Pclass` of 2.31 is a perfectly computed number describing nothing.

3.1 The two families and the four types



Type	Family	Definition	Titanic example
Continuous	Numerical	Any value in a range, including fractions. Between any two values there is a third.	`Age` (0.42 to 80), `Fare` (0 to 512.33)
Discrete	Numerical	Countable whole numbers. No value between 1 and 2.	`SibSp` (0–8), `Parch` (0–6)
Nominal	Categorical	Labels with no inherent order.	`Sex`, `Embarked`
Ordinal	Categorical	Labels with a meaningful order, but no meaningful distance.	`Pclass` ($1 > 2 > 3$)

The test for ordinal versus nominal: can you sort the values and defend the sort? `Pclass` sorts: first class outranks third. `Embarked` does not: Cherbourg is not greater than Queenstown. If you cannot defend the sort, it is nominal, and encoding it as 0, 1, 2 tells the model a lie it will act on.

The test for discrete versus continuous: is a value halfway between two observed values meaningful? An age of 28.5 is meaningful. A `SibSp` of 1.5 is not: nobody has one and a half

siblings aboard. In practice the distinction is looser than the textbook, because a discrete variable with many levels is treated as continuous and one with three levels is treated as categorical.

3.2 Why the classification drives everything downstream

Type	Visualise with	Impute with	Encode / scale with
Continuous	Histogram, KDE, boxplot, scatter	Median if skewed, mean if symmetric	Standardise or normalise; consider log or binning
Discrete	Countplot or histogram with integer bins	Median or mode	Often usable as-is; scale for distance-based models
Nominal	Countplot, bar chart, crosstab	Mode, or a literal `Missing` category	One-hot encoding
Ordinal	Countplot ordered by the category order	Mode	Ordinal encoding that preserves the order

The trap in the middle: a column can be numeric in dtype and categorical in meaning. `Pclass` arrives as `int64`. `df.corr()` will happily include it, `describe` will report its mean, and `StandardScaler` will z-score it. None of that is an error the computer can catch. `Survived` is the same: an `int64` column that is really a binary label.

3.3 Titanic, column by column

Column	pandas dtype	Real type	Consequence
`PassengerId`	int64	Identifier	Drop. A perfectly numeric column with zero information.
`Survived`	int64	Binary target	Never scale or impute it.
`Pclass`	int64	Ordinal	Do not take its mean. Ordinal-encode or one-hot.
`Name`	object	Text	Not a feature as-is; `Title` is extracted from it.
`Sex`	object	Nominal, 2 levels	Map to 0/1; one-hot adds nothing for a binary.
`Age`	float64	Continuous	Impute, then optionally bin.
`SibSp`	int64	Discrete	Combine with `Parch` into `FamilySize`.
`Parch`	int64	Discrete	As above.
`Ticket`	object	Identifier, partly	681 unique of 891; group size is the usable signal.
`Fare`	float64	Continuous	Heavily skewed. Median, and log or robust scaling.
`Cabin`	object	Nominal, 147 levels	77% missing; use presence and first letter, not the value.
`Embarked`	object	Nominal, 3 levels	Mode-impute the 2 missing, then one-hot.

3.4 Code

```
# What pandas guessed
df.dtypes

# Cardinality is the fastest categorical / continuous test
df.nunique().sort_values()
# Survived 2, Sex 2, Embarked 3, Pclass 3, SibSp 7, Parch 7,
# Age 88, Cabin 147, Fare 248, Ticket 681, Name 891, PassengerId 891

# Split the columns into working lists, once, and reuse them everywhere
numeric_cols     = ["Age", "Fare", "SibSp", "Parch"]
categorical_cols = ["Sex", "Embarked", "Pclass"]
identifier_cols  = ["PassengerId", "Name", "Ticket"]
target           = "Survived"

# Tell pandas what you mean. Category dtype also saves memory and
# fixes the plotting order for an ordinal.
df["Sex"]      = df["Sex"].astype("category")
df["Embarked"] = df["Embarked"].astype("category")
df["Pclass"]   = pd.Categorical(df["Pclass"], categories=[1, 2, 3], ordered=True)

# Inspect every categorical the same way
for col in categorical_cols:
    print(df[col].value_counts(dropna=False), "\n")
# Sex:      male 577, female 314
# Embarked: S 644, C 168, Q 77, NaN 2
# Pclass:   3 491, 1 216, 2 184
```

A heuristic worth automating: a numeric column with fewer than roughly 10–15 distinct values is probably categorical; one whose distinct count is close to the row count is probably an identifier. Neither rule is law, and both find the problems in a wide dataset far faster than reading the column names.

Common mistakes

- **Treating a numeric-looking category as a quantity.** Correlating `Pclass` or scaling `PassengerId` is the classic version.
- **Encoding a nominal variable as 0, 1, 2.** It asserts that Queenstown is halfway between Cherbourg and Southampton. Linear models believe that.
- **Destroying an ordinal's order.** One-hot encoding `Pclass` is defensible but throws away that first class outranks third; ordinal encoding keeps it.
- **Leaving identifiers in the feature matrix.** `PassengerId` correlates with nothing but can still be split on by a tree, producing pure noise memorised as signal.

Industry best practices

- Maintain the column lists (`numeric_cols`, `categorical_cols`) as **explicit variables** at the top of the script. Every transformer later refers to them, and a column that belongs to no list is a column you forgot.
- Use the **category dtype** for genuine categoricals. It is smaller, it makes plots order correctly, and it makes an unexpected value an error rather than a new group.
- Write a **data dictionary** and keep it with the code, not in someone's head.

- Validate types at the boundary with `pandera` or a handful of `assert` statements, so a schema change fails loudly.

Interview questions

1. **Difference between nominal and ordinal, with an example of each?**
   Both are categorical; ordinal has a defensible ordering. ***`Embarked`*** *is nominal,* ***`Pclass`*** *is ordinal.*

2. **`Pclass` is stored as an integer. Is it numerical?**
   In dtype yes, in meaning no. It is ordinal, so its mean is meaningless and it should be encoded rather than scaled.

3. **How would you decide whether a numeric column is really categorical?**
   Cardinality relative to row count, whether the values are codes, and whether arithmetic on them means anything.

4. **Why does the variable type change which plot you draw?**
   A histogram bins a continuous range; a countplot tallies discrete labels. Applying either to the wrong type produces a chart that cannot be read.

5. **What is the risk of label-encoding a nominal variable for a linear model?**
   It imposes a false order and false distances, and the model fits coefficients to that fiction.

4. Missing Value Analysis

Short description: A missing value is a cell with no observation. In pandas it is `NaN` for numerics and `None` or `NaN` for objects. Missing value analysis means measuring how much is absent, forming a theory about *why*, and choosing between dropping and filling.

4.1 Why missing values are dangerous

- **Most estimators refuse to run.** `LogisticRegression`, `SVC` and `StandardScaler` raise on `NaN`. The problem announces itself, which is the harmless case.
- **The silent case is worse.** `mean()` skips nulls, so a column that is 20% missing reports a mean over 80% of the rows without saying so.
- **Dropping rows destroys sample size.** On Titanic, `df.dropna()` leaves **183 of 891 rows**. A single 77%-missing column has taken 79% of the dataset with it.
- **Imputation shrinks variance.** Filling 177 ages with one value pulls the standard deviation down and the peak up. Every confidence interval computed afterwards is too narrow.
- **Missingness can be the signal.** On Titanic, passengers with a recorded `Cabin` survived at **66.67%** against **29.99%** for those without. That gap is not about cabins; it is about who the records were kept for. Throwing the column away throws that away too.

4.2 The three mechanisms

Why a value is missing determines whether filling it is honest.

Mechanism	Meaning	Consequence
MCAR, missing completely at random	The probability of being missing is unrelated to anything, observed or not. A sensor dropped packets.	Safest case. Dropping rows is unbiased, imputation is unbiased. Rare in practice.
MAR, missing at random	Missingness depends on *other observed* columns. Age is recorded less often in third class.	Imputation works, provided you condition on those columns: median age *per class and title*, not one global median.
MNAR, missing not at random	Missingness depends on the unobserved value itself. High earners decline to state income.	No imputation is unbiased. Model the missingness explicitly with an indicator column and say so.

On Titanic: `Cabin` is close to MNAR, since records survived for wealthier passengers, which is exactly what `HasCabin` captures. `Age` looks MAR: it is predictable from `Title` and `Pclass`, which is why the group-wise median beats the global one. `Embarked`, with two missing rows, can be treated as MCAR because at 0.22% the mechanism cannot change any conclusion.

4.3 Finding them

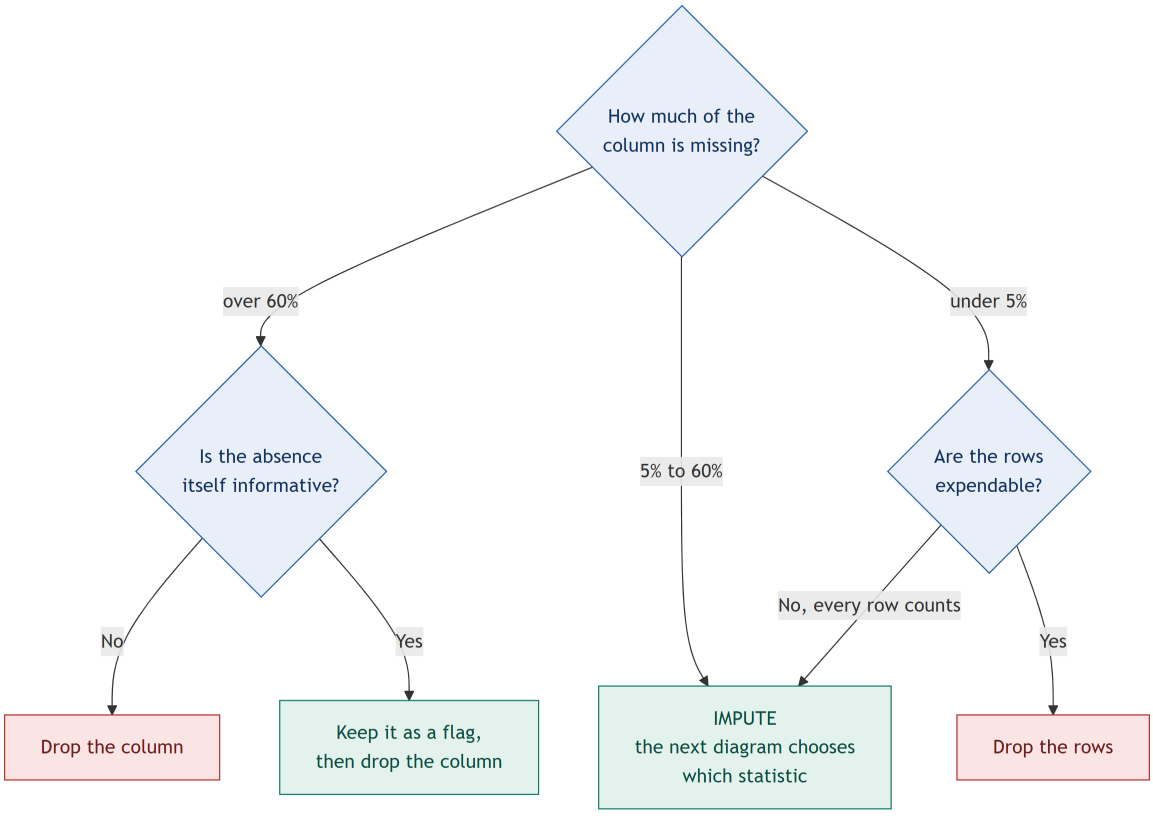
```python
df.isnull().sum()                       # count per column
df.isnull().sum()[lambda s: s > 0]      # only the columns that have any
(df.isnull().mean() * 100).round(2)     # percentage per column
df.isnull().sum().sum()                 # total cells missing
df.isnull().any(axis=1).sum()           # rows with at least one missing value

# A one-line missingness map: white stripes are the gaps
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap="viridis")
plt.title("Missing values by column")
plt.show()

# Is the missingness informative? Compare the target across the gap.
df.groupby(df["Cabin"].isnull())["Survived"].agg(["mean", "count"])
# False (cabin recorded): 0.6667 over 204 rows
# True  (cabin missing) : 0.2999 over 687 rows
```

Column	Missing	%	Decision, and why
`Cabin`	687	77.10	Drop the column, but keep `HasCabin` and `Deck` first. Too sparse to impute, too informative to discard silently.
`Age`	177	19.87	Impute. 20% is far too much to drop, and `Age` is predictive. Median, grouped by `Title` and `Pclass`.
`Embarked`	2	0.22	Impute with the mode (`S`), or drop the two rows. At this rate the choice cannot matter.

4.4 The decision guide



When to DROP a column:

- More than roughly 60–70% missing, *and* the presence pattern carries no signal you have extracted.
- The column is an identifier, a constant, or a near-duplicate of another column.
- Domain knowledge says the values are unreliable where they do exist.
- **Titanic:** `Cabin`, at 77.10% missing, but only after deriving `HasCabin` and `Deck` from it.

When to DROP rows:

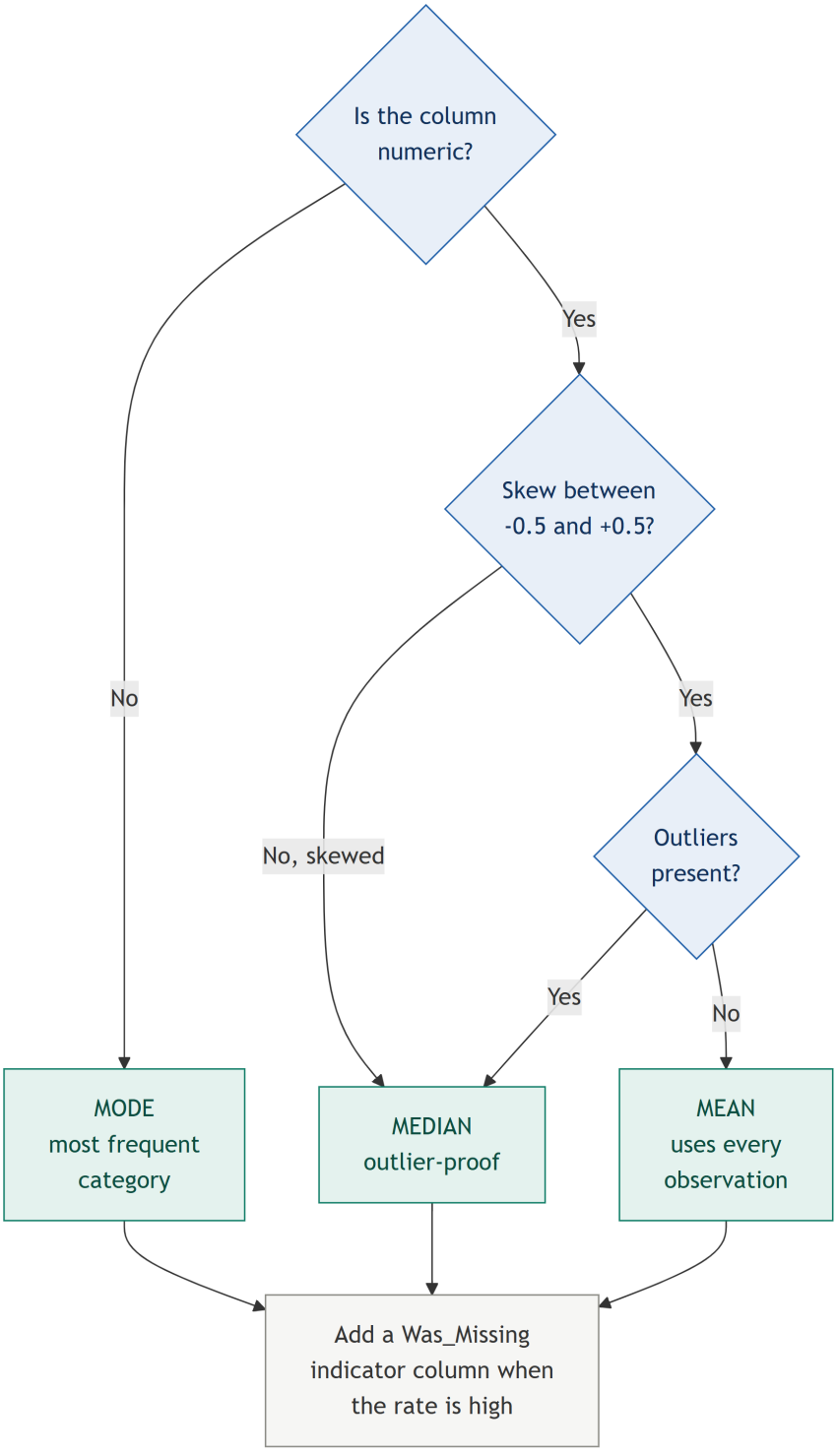
- The affected rows are a very small fraction, under roughly 5%.
- The target itself is missing. Never impute the target; a row with no label is not a training example.
- The row is broken across several columns, which usually means a parsing failure rather than a real observation.
- **Titanic:** the two rows with no `Embarked`, if you prefer dropping to mode-filling.
- **Never** apply a blanket `df.dropna()` before checking what it costs. Here it costs 708 of 891 rows.

When to IMPUTE:

- Between roughly 5% and 60% missing, in a column you have reason to keep.
- The dataset is small enough that rows are expensive. 891 rows is small.
- The mechanism is MCAR or MAR, so a conditional estimate is defensible.
- **Titanic:** `Age` at 19.87%.

The thresholds are heuristics, not rules. A column that is 90% missing but records whether a customer contacted support is worth keeping as a flag. A column that is 10% missing and duplicates another is worth dropping. The percentage starts the conversation; what the column means finishes it.

4.5 Mean, median and mode imputation



Mean imputation

Concept: Replace every missing value with the arithmetic mean of the observed values. The column mean is unchanged by construction, which is its one guarantee.

When to use: Numeric columns whose distribution is roughly symmetric, with a skew between about -0.5 and $+0.5$, and no heavy outliers. Interval measurements such as temperature or a standardised test score.

When NOT to use:

- Skewed distributions. The mean sits inside the tail, so every imputed row is pulled away from the bulk of the data.
- Any column with strong outliers. One `Fare` of 512.33 lifts the mean of that column from a median of 14.45 to 32.20; imputing with 32.20 invents 15 fictional first-class-priced tickets.
- Categorical variables, where an average has no meaning.
- Bounded counts where the mean is not attainable. An imputed `SibSp` of 0.52 is not a possible number of siblings.

Median imputation

Concept: Replace missing values with the 50th percentile: the value with half the observations below it. It depends only on the rank of the data, not on the magnitude of the extremes.

When to use: Skewed distributions; any column with outliers; monetary amounts, durations, counts, anything bounded below by zero and unbounded above. When in doubt on a numeric column, this is the safe default.

When NOT to use: Categorical variables. Also, be careful on a strongly bimodal column: the median can land in the empty valley between the two modes, producing imputed values that resemble nothing in the data.

Mode imputation

Concept: Replace missing values with the most frequent value.

When to use: Categorical variables, nominal or ordinal, and only when the missing rate is low. `Embarked` on Titanic is the ideal case: two missing rows, and `S` accounts for 644 of the 889 observed.

When NOT to use: When the missing rate is high. Filling 40% of a column with its most common category manufactures a majority that was never observed and flattens the column's usefulness. Above roughly 10%, prefer an explicit `Missing` category, which is honest and lets the model decide whether absence matters.

	Mean	**Median**	**Mode**
Applies to	Numeric	Numeric	Categorical, or numeric with few levels
Outlier sensitivity	High	None	None
Needs	Symmetry	Nothing	Low missing rate
Preserves	The column mean	The column median	The dominant category
Damages	The distribution shape when skewed	The variance and the tails	The class balance when overused
Titanic use	Not used	`Age`	`Embarked`

4.6 Skewness: the number that decides mean versus median

Concept: Skewness measures the asymmetry of a distribution. It is the third standardised moment: the average cubed z-score. Cubing preserves sign, so values far above the mean contribute large positive terms and values far below contribute large negative ones.

$$\text{skew} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^3$$

```
df["Age"].skew()      # 0.3891  -> mild right skew, close to symmetric
df["Fare"].skew()     # 4.7873  -> extreme right skew
df["SibSp"].skew()    # 3.6954
df["Parch"].skew()    # 2.7491

np.log1p(df["Fare"]).skew()   # 0.3949 -- one transform fixes it
```

Skew value	Shape	What to do
Exactly 0	Perfectly symmetric; mean = median	Mean imputation is safe. Rare in real data.
-0.5 to $+0.5$	Approximately symmetric	Mean or median; both are defensible. **Age at 0.39 sits here.**
$+0.5$ to $+1$	Moderate right skew	Prefer the median. Consider a square-root transform.
Greater than $+1$	Strong right skew; a long tail of large values; mean > median	Median imputation, and transform the column: `log1p` or Box-Cox. **Fare at 4.79 sits here.**
-0.5 to -1	Moderate left skew	Median.
Less than -1	Strong left skew; long tail of small values; mean < median	Median, and consider a squared or exponential transform.

Read it against the histogram, not instead of it. Skewness is one number and a histogram is the whole shape. A perfectly symmetric bimodal distribution has a skew of zero and should never be mean-imputed. The number is a filter for a wide dataset; the plot is the decision.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(df["Age"], kde=True, bins=30, ax=axes[0])
axes[0].axvline(df["Age"].mean(),   color="red",   label=f"mean {df['Age'].mean():.1f}")
axes[0].axvline(df["Age"].median(), color="green", label=f"median {df['Age'].median():.1f}")
axes[0].set_title(f"Age  (skew = {df['Age'].skew():.2f})")
axes[0].legend()
```

```
sns.histplot(df["Fare"], kde=True, bins=40, ax=axes[1])
axes[1].axvline(df["Fare"].mean(),   color="red",   label=f"mean {df['Fare'].mean():.1f}")
axes[1].axvline(df["Fare"].median(), color="green", label=f"median {df['Fare'].median():.1f}")
axes[1].set_title(f"Fare (skew = {df['Fare'].skew():.2f})")
axes[1].legend()
plt.tight_layout(); plt.show()
```

What the two histograms show: `Age` is a single hump centred near 28 with a small bulge of children at the left and a thin tail to 80; the mean line at 29.70 and the median at 28.00 nearly coincide. `Fare` is a wall at the left, half the passengers under 14.45 pounds, with a tail running out to 512.33 and the mean line dragged out to 32.20, well clear of the median. The same chart makes both imputation decisions.

4.7 Titanic: the three decisions, with numbers

`Age` to the median, not the mean. Skew is 0.39, technically in the symmetric band, so why the median? Because the mean of 29.70 is being pulled by the same tail that makes a small difference here and a large difference in any resample, and because the median costs nothing when the two nearly agree. The measured effect of filling with the median:

Age column	mean	std	skew
Observed only, 714 rows	29.70	14.53	0.389
Filled with the global median 28.0	29.36	13.02	0.510
Filled with the global mean 29.70	29.70	13.00	0.434
Filled with the median per `Title` × `Pclass`	29.14	13.49	0.474

Read that table carefully, it contains the real lesson. Every imputation drops the standard deviation, from 14.53 to about 13. That is variance you invented away by placing 177 rows on one point. The group-wise fill loses the least of it (13.49) because it spreads the imputed values across five titles and three classes instead of stacking them on one value. No imputation is free; the goal is to choose the one that lies least.

`Embarked` to the mode. Two rows, 0.22%. `S` covers 644 of 889 observed values. Filling with `S` is the least assumptive thing available and cannot move any result. Dropping the two rows is equally defensible.

`Cabin` dropped, after extraction. 687 of 891 missing. There is no honest way to impute a cabin number. But before dropping: `HasCabin` separates 66.67% survival from 29.99%, and `Deck`, the first letter, has 147 distinct cabins collapsing to 8 decks (C 59, B 47, D 33, E 32, A 15, F 13, G 4, T 1). Extract, then drop the raw column.

4.8 Code, from naive to production

```
# --- Level 1: the naive version. Works, and leaks. ---------------------------
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df = df.drop(columns=["Cabin"])

# --- Level 2: group-wise, which respects MAR ---------------------------------
df["Title"] = df["Name"].str.extract(r",\s*([^\.]+)\.")[0].str.strip()
df["Age"] = df["Age"].fillna(
    df.groupby(["Title", "Pclass"])["Age"].transform("median")
)
```

```python
df["Age"] = df["Age"].fillna(df["Age"].median())   # backstop for empty groups

# --- Level 3: sklearn, so the same statistic is reused on unseen data --------
from sklearn.impute import SimpleImputer

num_imputer = SimpleImputer(strategy="median")
cat_imputer = SimpleImputer(strategy="most_frequent")

X_train[["Age"]] = num_imputer.fit_transform(X_train[["Age"]])  # learns here
X_test[["Age"]]  = num_imputer.transform(X_test[["Age"]])       # reuses it

# --- Keeping the fact that a value was missing --------------------------------
df["Age_was_missing"] = df["Age"].isnull().astype(int)   # BEFORE filling
df["HasCabin"] = df["Cabin"].notna().astype(int)
df["Deck"] = df["Cabin"].str[0].fillna("Unknown")
```

The order in the last block matters. `Age_was_missing` has to be computed before the fill, or it is all zeros. This is the single most common bug in imputation code.

4.9 Beyond mean, median and mode

Method	How it works	When it earns its cost
Group-wise statistic	Median within groups defined by other columns	Whenever the mechanism is MAR. Cheap, interpretable, usually enough.
`KNNImputer`	Averages the k nearest complete rows in feature space	Correlated numeric features, moderate dataset size. Needs scaled inputs and is slow above roughly 100k rows.
`IterativeImputer`	Regresses each column on the others, in rounds (the MICE idea)	Several interrelated numeric columns and enough data to fit those regressions.
Constant sentinel	Fill with `Missing` or -1	Categoricals, and tree models, which can split on the sentinel and learn what absence means.
Forward / backward fill	Carry the previous observation forward	Time series only. Applying it to unordered rows fabricates order that is not there.
Model-native handling	LightGBM and XGBoost learn a default direction for `NaN`	When you are using them. Often better than anything you would impute by hand.

Do not reach for the sophisticated option first. On Titanic, a grouped median beats a global median and `KNNImputer` adds essentially nothing over it for 177 rows. The complexity of the imputer should track the amount and importance of what is missing.

Common mistakes

- **Imputing before splitting.** The median of the full dataset contains the test rows. It is a small leak, it is always avoidable, and it is the one interviewers ask about.

- **Blanket `dropna()`.** On Titanic that is 891 rows down to 183. Count the cost before you pay it.
- **Mean-imputing a skewed column.** `Fare` filled with 32.20 rather than 14.45 is the textbook version.
- **Imputing the target.** A row with no label is not a training example.
- **Dropping a high-missing column without checking the pattern.** `Cabin`'s absence is worth 37 percentage points of survival rate.
- **Filling with 0 because it is numeric.** A fare of 0 and an unknown fare are different facts. Fifteen passengers really did pay 0.
- **Forgetting the indicator column.** If 20% of a column is imputed, the model deserves to know which rows.

Industry best practices

- Put every imputer inside a `Pipeline`. It makes the leak structurally impossible rather than a thing you remembered not to do.
- **Record the imputation values** that were fitted. When production predictions drift, the first question is whether the input distribution moved away from them.
- Add `add_indicator=True` to `SimpleImputer` for any column above roughly 10% missing. It is one argument and it preserves the signal.
- **Monitor the missing rate in production.** A column that was 2% missing in training and is 40% missing today is an upstream incident, not a modelling problem.
- **Write down the mechanism** you assumed, MCAR, MAR or MNAR, and why. It is the assumption most likely to be wrong and least likely to be recorded.

Interview questions

1. **MCAR, MAR and MNAR: define them and give an example of each.**
   As in the table above. The practical point is that only MNAR resists honest imputation, and it needs an explicit indicator instead.
2. **Why median rather than mean for `Age`?**
   The median is unaffected by the tail, costs nothing when the two agree, and cannot be dragged by outliers in a resample.
3. **When would you drop a column instead of imputing it?**
   Above roughly 60–70% missing, once you have extracted any signal in the missingness pattern itself.
4. **Why does imputation reduce variance, and does it matter?**
   Every imputed row sits exactly on the statistic, so the spread shrinks: on Titanic 14.53 to 13.02. It matters because standard errors and confidence intervals computed afterwards are too narrow.
5. **Your column is 30% missing and the missingness predicts the target. What do you do?**
   Keep an explicit indicator, impute the values with a group-wise or model-based estimate, and treat the mechanism as MNAR in the write-up.

6. **Why must the imputer be fitted on the training set only?**
   Otherwise the statistic encodes test-set information, the test score is optimistic, and production performance does not match it.

5. Duplicate Data

Short description: A duplicate is a row that repeats an observation already present. The word hides two very different things: an *exact* duplicate, where every column matches, and a *logical* duplicate, where the same real-world entity appears twice with different surface values.

Where duplicates come from: a join that fanned out because the key was not unique; an ETL job rerun after a partial failure; a form submitted twice; concatenating files that overlap at the boundary; a scraper that revisited a page. Almost none of these are visible in the data itself. That is why you check rather than assume.

5.1 Why duplicates harm a model

- **They reweight the training data.** A row appearing three times counts three times in the loss. The model is being told, without evidence, that this pattern is three times as common as it is.
- **They corrupt every statistic you computed.** Means, correlations, class balance and the missing-value percentages all shift.
- **They leak across the split.** This is the serious one. If a row exists twice and the split sends one copy to train and one to test, the model has memorised a test example. The test score is then partly a measure of memory, and it will not reproduce in production.
- **They inflate confidence.** Standard errors assume independent observations. Duplicates are perfectly dependent, so every interval comes out too narrow.

5.2 Detection

```
df.duplicated().sum()          # exact duplicates across every column -> 0 on Titanic
df[df.duplicated(keep=False)]  # show every copy, not just the later ones

# The check that actually finds things: ignore surrogate keys first
df.drop(columns=["PassengerId"]).duplicated().sum()      # 0

# Logical duplicates: same person, different row
df.duplicated(subset=["Name", "Sex", "Age"]).sum()       # 0

# The near-duplicate check, for text keys that differ by whitespace or case
key = df["Name"].str.lower().str.replace(r"\s+", " ", regex=True).str.strip()
key.duplicated().sum()                                   # 0
```

Titanic result: zero duplicates, by every one of those definitions. That is worth stating rather than skipping: the check is two seconds and the alternative is assuming.

The check that is nearly always wrong: `df.duplicated()` on a table with a surrogate key. `PassengerId` is unique by construction, so no two rows can ever be exact duplicates, and the count is guaranteed to be zero whether or not the data is clean. Drop the identifier columns first, or pass an explicit `subset` of the columns that define a real-world entity.

5.3 Removal

```
df = df.drop_duplicates()                                  # keep the first copy
df = df.drop_duplicates(subset=["Name", "Age"], keep="last")  # keep the most recent
df = df.drop_duplicates(subset=["Name", "Age"], keep=False)   # drop every copy
```

```python
# After dropping rows, the index has holes. Reset it.
df = df.drop_duplicates().reset_index(drop=True)

# Log what you removed. A silent -4000 rows is a bug report waiting to happen.
before = len(df)
df = df.drop_duplicates()
print(f"removed {before - len(df)} duplicate rows ({before} -> {len(df)})")
```

On `inplace=True`: the guide's earlier code used `df.drop_duplicates(inplace=True)` because that is the form everyone writes first. Prefer reassignment. `inplace` does not save memory in modern pandas, it makes the cell non-idempotent, and it is deprecated in several places. Reassigning means you can rerun the cell.

5.4 When NOT to remove duplicates

Situation	Why the repeat is real
Transactions	Two customers buying the same item, at the same price, in the same minute, produce identical rows. Both happened.
Time series and event logs	The same reading at two timestamps is a measurement, not a copy. Check the timestamp before deduplicating.
Aggregated or panel data	One row per person per month legitimately repeats the person's static attributes.
Deliberate oversampling	If you resampled a minority class on purpose, the repeats are the technique.
Survey and census data	Two respondents can genuinely give identical answers to a short questionnaire.
Low-cardinality tables	With five binary columns there are only 32 possible rows; repeats are arithmetic, not error.

The rule: a duplicate is a row that should not exist *given what one row is supposed to represent.* Answer that question first, in words, and the deduplication rule writes itself. On Titanic one row is one passenger, so `Name` plus `Age` is a reasonable identity and `PassengerId` is not.

The Titanic case that looks like a duplicate and is not: 681 unique tickets across 891 passengers means 210 people share a ticket with somebody. Those rows repeat the `Ticket`, the `Fare` and often the `Cabin`. They are not duplicates, they are families, and Section 10 turns exactly that repetition into `TicketGroup` and `FarePerPerson`.

Common mistakes

- **Running `duplicated()` with an ID column still present** and concluding the data is clean.
- **Deduplicating after the split.** Do it before, or copies land on both sides.
- **Removing duplicates in time-series data** where repetition is the measurement.
- **Not logging the count.** You need to know whether you dropped 4 rows or 40,000.
- **Forgetting `reset_index(drop=True)`**, then being confused when `iloc` and `loc` disagree.

Industry best practices

- Define a **business key** for the table and check uniqueness on it, not on all columns. The key is a modelling decision, not a technicality.
- Deduplicate **as early as possible**, ideally in the query or ingestion job, so no downstream statistic is ever computed on the inflated table.
- Make it an **assertion in the pipeline**, not a manual step: `assert df.duplicated(subset=KEY).sum() == 0`.
- Investigate a **sudden change in the duplicate rate**. It usually means an upstream job ran twice, and the fix belongs there.

Interview questions

1. **Why are duplicates a problem for model training?**
   They reweight the loss, distort every summary statistic, and can leak the same row into both train and test, which makes the test score partly a memory test.
2. **`df.duplicated().sum()` returns 0. Is the data clean?**
   Not necessarily. With a surrogate key present the answer is always 0. Drop identifiers or pass an explicit subset.
3. **Give a case where duplicates must be kept.**
   Transaction records, sensor readings, panel data, or a deliberately oversampled minority class.
4. **Should you deduplicate before or after the train/test split?**
   Before, otherwise copies of one row can end up on both sides.
5. **How do you find near-duplicates rather than exact ones?**
   Normalise the key columns (case, whitespace, punctuation) and compare; for free text, use fuzzy matching or a hash of the normalised string.

6. Outlier Analysis

Short description: An outlier is an observation far from the bulk of the data. Far is not a fact about the point; it is a statement about the distribution you are comparing it to. The same fare of 512 pounds is an extreme outlier among all passengers and an unremarkable value among first-class suite bookings.

6.1 Why outliers matter

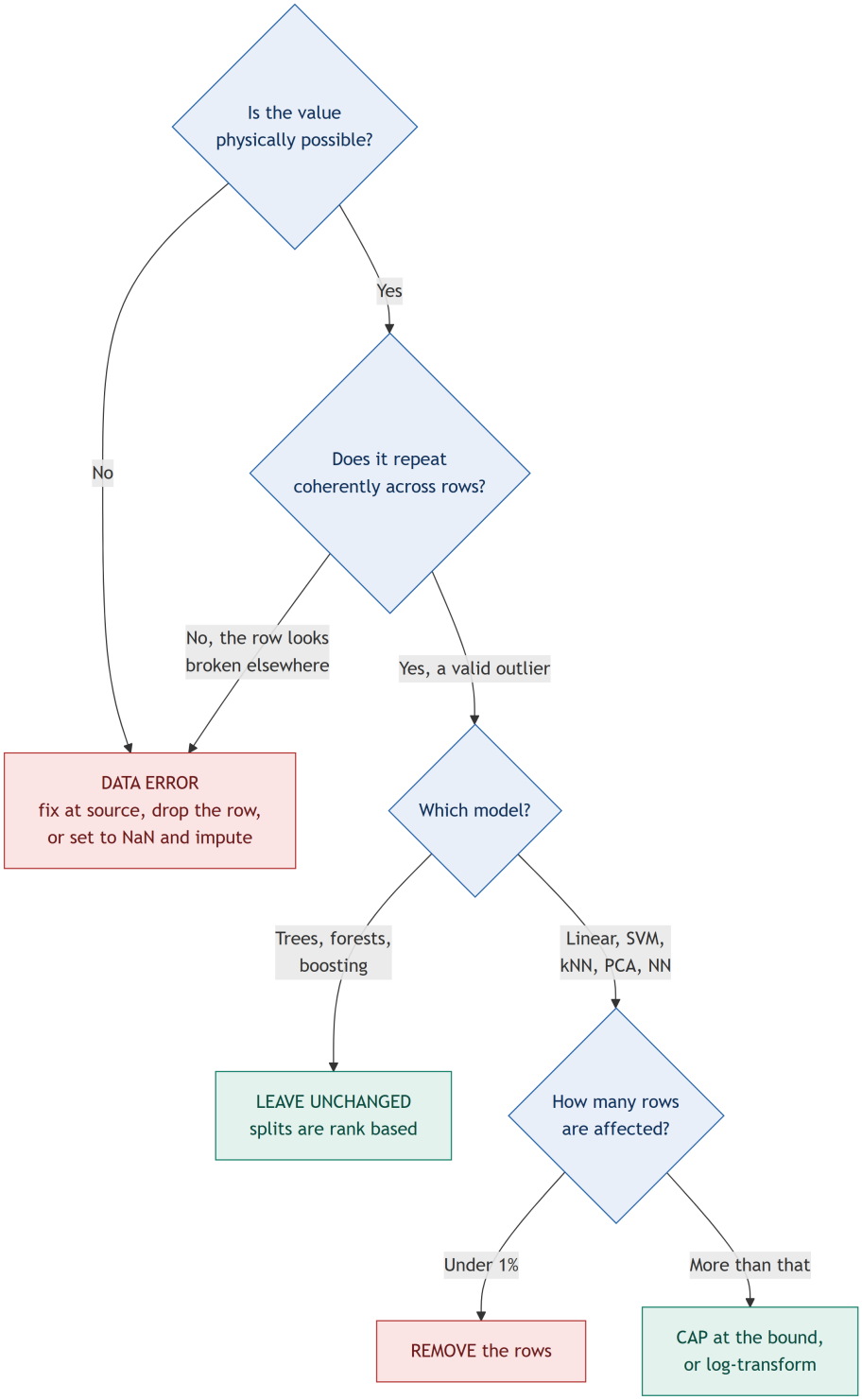
What they damage	Mechanism
Summary statistics	The mean and standard deviation are defined by magnitude. `Fare` has a median of 14.45 and a mean of 32.20; the gap is almost entirely a few large values.
Linear models	Least squares minimises *squared* error, so a point ten units off contributes a hundred times the pull of a point one unit off. One extreme row can tilt the whole fit.
Distance-based models	kNN, K-Means and SVM measure distance. An unscaled outlier dominates every distance it appears in.
Scalers	`StandardScaler` uses the mean and std, `MinMaxScaler` uses the min and max. Both are defined by the values you are worried about.
Neural networks	Large inputs produce large gradients and unstable training.
Correlation	Pearson's r is computed from the same moments; a single point can create or destroy an apparent relationship.

What they do not damage: tree-based models. A decision tree splits on *rank*, asking whether `Fare` exceeds a threshold, not by how much. Moving a value from 512 to 5000 changes nothing about which side of any split it falls on. Random forests and gradient boosting inherit this. If your final model is a tree ensemble, outlier treatment is frequently unnecessary work.

6.2 Valid outlier or data error?

This is the first question and it is not statistical. It is a domain question, and the answer decides everything downstream.

	Data error	Valid outlier
Definition	The value is impossible or corrupt	The value is real and rare
Cause	Sensor fault, unit mix-up, typo, parsing failure, sentinel value like −999	Genuine variation in the population
Test	Could this exist in the world?	Does it repeat coherently across rows?
Action	Fix at source, set to `NaN` and impute, or drop the row	Keep, cap, or transform. Never silently delete.
Titanic example	An `Age` of 250, or a negative `Fare`. Neither occurs in this file.	`Fare` = 512.33, three real first-class passengers.



The Titanic case, worked: the three highest fares are all exactly 512.3292 and all share the ticket number `PC 17755`. That coherence is the tell. It is not a data entry error repeated three times; it is one expensive ticket covering three people, one of whom occupied cabins B51, B53 and B55. All three survived. The right response is not to delete them but to build `FarePerPerson`, which turns 512.33 into 170.78 and removes half the outliers in the column outright.

The other end of the same column: fifteen passengers have a `Fare` of exactly 0. That is not a missing value coded as zero; those are crew, guests of the line and staff travelling on duty. Imputing them would erase a real category.

6.3 The boxplot

```
sns.boxplot(x=df["Fare"])
plt.title("Fare distribution")
plt.show()

# Split by a category to see whether the outliers are one group
sns.boxplot(x="Pclass", y="Fare", data=df)

# For a skewed variable, a log axis makes the box readable
sns.boxplot(x=df["Fare"]); plt.xscale("log")
```

Element	What it is, on `Fare`
Left edge of box	Q1, the 25th percentile. 7.91
Line inside box	Q2, the median. 14.45
Right edge of box	Q3, the 75th percentile. 31.00
Box width	The IQR, 23.09. The middle half of the passengers.
Whiskers	Extend to the furthest point within 1.5 IQR of the box: to 65.63 on the right, and to 0 on the left because the computed lower bound is negative.
Dots beyond	Flagged as outliers. There are 116 of them on the right.

Why the boxplot is the right first chart: it shows the centre, the spread, the skew and the outliers in one object, and it is unaffected by them while doing so, because every element is a rank. A histogram of `Fare` is a wall at zero and a flat line; the boxplot is legible.

6.4 The IQR method

Concept: Define the normal range from quartiles, which are rank-based and therefore unmoved by the extremes they are being used to detect. Anything outside that range is flagged.

Quantity	Definition	`Fare`
Q1	25th percentile: 25% of values are below it	7.9104
Q3	75th percentile: 75% of values are below it	31.0000
IQR	$Q3 - Q1$, the spread of the middle half	23.0896
Lower bound	$Q1 - 1.5 \times IQR$	-26.7240
Upper bound	$Q3 + 1.5 \times IQR$	65.6344

$$IQR = Q3 - Q1 \qquad \text{Lower} = Q1 - 1.5 \times IQR \qquad \text{Upper} = Q3 + 1.5 \times IQR$$

Where 1.5 comes from: Tukey's choice. For a normal distribution the fences sit at about ± 2.7 standard deviations, so roughly 0.7% of normal data falls outside by chance. It is a convention, not a law. Use 3.0 instead of 1.5 for the far-out fence: on `Fare` that cuts the flagged count from 116 to 53.

```
Q1  = df["Fare"].quantile(0.25)     # 7.9104
Q3  = df["Fare"].quantile(0.75)     # 31.0
```

```
IQR = Q3 - Q1                          # 23.0896

lower = Q1 - 1.5 * IQR                 # -26.724
upper = Q3 + 1.5 * IQR                 #  65.6344

mask     = (df["Fare"] < lower) | (df["Fare"] > upper)
outliers = df[mask]
print(len(outliers), f"{len(outliers) / len(df):.2%}")   # 116, 13.02%

# Always ask what the flagged rows have in common
outliers["Pclass"].value_counts()          # overwhelmingly first class
outliers["Survived"].mean()                # 0.681 vs 0.384 overall
```

That last line is the whole argument against deleting them. The flagged rows survive at 68.1% against a baseline of 38.4%. Those 116 rows are not noise; they are 13% of the dataset carrying the strongest single signal in the fare column. Deleting them would delete the relationship you are trying to model.

A note on `Age`: applying the same method gives Q1 = 20.125, Q3 = 38.0, IQR = 17.875, and an upper bound of 64.81. Eleven passengers are flagged, all of them simply elderly, the oldest being 80. There is nothing to treat. The method flags; you decide.

6.5 Other detection methods

Method	Rule	Notes
IQR / Tukey	Outside $Q1 - 1.5\,IQR$ to $Q3 + 1.5\,IQR$	Distribution-free, robust. The default. 116 flagged on `Fare`.
Z-score	$\lvert z\rvert > 3$ where $z = (x - \bar{x})/s$	Assumes approximate normality, and the outliers inflate s, so extreme points mask each other. Flags only 20 on `Fare`, fewer than IQR precisely because of that inflation.
Modified z-score	Uses the median and the median absolute deviation	The robust version of the z-score. Prefer it if you want a z-style rule.
Percentile capping	Clip below the 1st and above the 99th percentile	Blunt, fast, and honest about being arbitrary.
Isolation Forest	Learns which points are easy to isolate	Multivariate: catches a row that is unremarkable in every column but impossible in combination, such as a 5-year-old travelling alone in first class.
Local Outlier Factor	Compares local density to neighbours	Multivariate, good for clustered data.

Univariate methods miss combinations. A `Fare` of 30 is ordinary and an `Age` of 4 is ordinary; a 4-year-old who booked their own third-class ticket at 30 pounds is not. Only a multivariate method sees that, which is why Isolation Forest is worth knowing even though the IQR rule handles most single columns.

6.6 Remove, cap or leave

When to REMOVE the rows:

- The value is a confirmed **data error** and cannot be corrected.
- The affected rows are a **tiny fraction**, under about 1%, and the dataset is large enough not to miss them.
- The row is broken in several columns at once.
- **Not applicable to Titanic's `Fare`.** 13% is not a tiny fraction, and the rows are real.

When to CAP (winsorize):

- The values are **real but extreme**, and you need to limit their influence without discarding the observations.
- You are using a model that is sensitive to magnitude and you cannot afford to lose rows.
- The variable has a natural ceiling that the data exceeds through noise.
- **Titanic:** capping `Fare` at 65.63 keeps all 891 passengers and stops three tickets dominating a linear model's coefficient.

When to LEAVE them alone:

- You are training **tree-based models**. They do not care.
- The outliers **are the thing you are predicting**: fraud, equipment failure, rare disease. Deleting them deletes the positive class.
- The rows are real and you would rather **transform** the column than clip it.
- You cannot justify the boundary. An arbitrary cut you cannot defend is worse than leaving the data intact.

6.7 Winsorization, capping and transformation

Concept: Winsorizing replaces every value beyond a bound with the bound itself. The row survives, its rank order survives, and only its magnitude is limited. It is named after Charles Winsor and it is the least destructive intervention available.

```
# 1. Cap at the IQR fences
df["Fare_capped"] = df["Fare"].clip(lower=lower, upper=upper)
# max drops from 512.33 to 65.63; no rows are lost

# 2. Cap at percentiles instead, if you prefer a fixed proportion
lo, hi = df["Fare"].quantile([0.01, 0.99])
df["Fare_capped"] = df["Fare"].clip(lo, hi)

# 3. Transform rather than clip. Usually the best answer for money.
df["Fare_log"] = np.log1p(df["Fare"])   # log1p handles the 15 zero fares
df["Fare_log"].skew()                   # 4.7873 -> 0.3949

# 4. Fix the cause, not the symptom
tickets = df["Ticket"].map(df["Ticket"].value_counts())
df["FarePerPerson"] = df["Fare"] / tickets
# the 512.33 ticket becomes 170.78; flagged outliers fall from 116 to 59

# 5. Remove, when and only when the values are errors
df = df[(df["Fare"] >= lower) & (df["Fare"] <= upper)]
```

Treatment	Effect on `Fare`	Rows kept	Skew
None	Range 0 to 512.33	891	4.79
Cap at 1.5 IQR	Range 0 to 65.63	891	1.08
`log1p`	Range 0 to 6.24	891	0.39
`FarePerPerson`	Range 0 to 221.78, 59 flagged	891	4.38
Remove flagged rows	Range 0 to 65.00	775	1.43

The ranking for this column: `log1p` wins. It keeps every row, keeps the full ordering of fares, and takes the skew from 4.79 to 0.39 in one operation, which also happens to be exactly what a linear model wants. Capping is second. Deletion is last, and costs 116 of the most informative rows in the dataset.

Common mistakes

- **Deleting outliers reflexively.** On Titanic that discards 13% of the data and the rows that survived at 68%.
- **Detecting outliers on the full dataset before splitting.** The bounds are statistics; fitting them on test rows is leakage.
- **Using the z-score on a heavily skewed column.** The outliers inflate the standard deviation and hide each other: 20 flagged by z-score against 116 by IQR.
- **Treating outliers before deciding whether the model cares.** For a random forest this is unpaid work.
- **Removing the positive class.** In fraud, churn or fault detection the outliers are the target.
- **Applying a cap fitted on train to test with a different formula.** Store the bound; apply the stored number.

Industry best practices

- **Investigate before you treat.** Look at the flagged rows as rows. Fifteen zero fares and three shared suites are facts, not noise.
- **Prefer transformation to deletion**, and capping to both, when the values are real.
- **Fit the bounds on the training set** and store them as model artefacts, the same as any other learned parameter.
- **Set domain-based bounds where they exist.** Age between 0 and 120 is a business rule and beats any quantile.
- **Monitor the outlier rate in production.** A jump usually means an upstream unit change or a broken sensor, not a change in the population.
- **Report both scores** when you do treat outliers: with and without. If the treatment is doing the work, you want to know.

Interview questions

1. **Define the IQR method and state the formula.**
   $IQR = Q3 - Q1$; flag anything below $Q1 - 1.5\,IQR$ or above $Q3 + 1.5\,IQR$. Rank-based, so the extremes cannot hide themselves.

2. **Why is the IQR method preferred over the z-score for skewed data?**
   Quartiles are unaffected by extreme values; the mean and standard deviation are defined by them, so outliers inflate the denominator and mask each other.

3. **You find 13% of rows flagged as outliers. Do you delete them?**
   No. 13% is a population, not an anomaly. Check whether they are coherent, check their relationship with the target, and prefer transforming or capping.

4. **Distinguish a valid outlier from a data error.**
   Ask whether the value is physically possible and whether it repeats coherently. Errors get fixed, nulled or dropped; valid outliers get kept, capped or transformed.

5. **Which algorithms are robust to outliers?**
   Tree-based ones, because they split on rank. Linear models, SVM, kNN, K-Means and neural networks are not.

6. **What is winsorization?**
   Replacing values beyond a chosen bound with the bound itself, limiting influence while keeping every row and the full ordering.

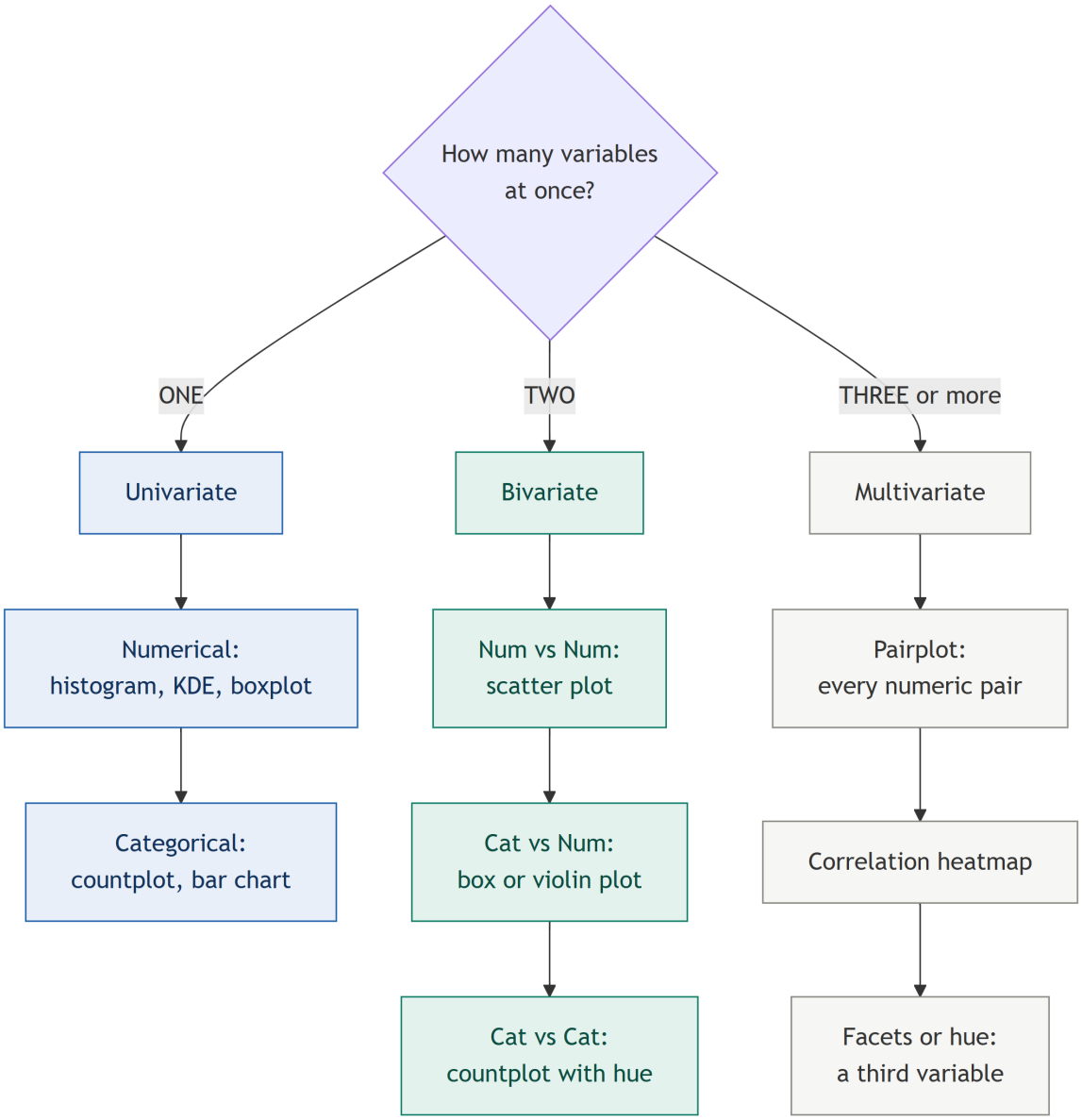
7. **How would you detect an outlier that is normal in every single column?**
   A multivariate method: Isolation Forest, Local Outlier Factor, or Mahalanobis distance.

7. Univariate Analysis

Definition: Analysis of one variable at a time. *Uni* for one, *variate* for variable. No relationships yet, no target yet: just the shape of each column on its own.

Goal: Establish, for every column, its centre, its spread, its shape, its extremes and its gaps. Univariate analysis is where you decide the imputation strategy, spot the skew that dictates the scaler, notice the category with three observations that will break your one-hot encoding, and find the values that are impossible.

Why it comes first: a relationship between two variables cannot be interpreted until you know what each variable looks like alone. A correlation of 0.26 between `Fare` and `Survived` means one thing if `Fare` is symmetric and quite another if half its mass sits below 14.45 with a tail to 512.



7.1 Numerical variables

Chart	What it shows	What it hides
Histogram	The distribution's shape: modes, gaps, skew, the range where the mass sits	Depends entirely on the bin count. Too few bins smooth a second peak away; too many turn the data into noise.
KDE plot	A smoothed density curve, easier to compare between groups	The smoothing is an assumption. It invents density between isolated points and spills past hard boundaries, showing negative ages.
Box plot	Median, quartiles, spread and outliers, all rank-based	The shape. Two very different distributions can produce identical boxes; a bimodal column looks unremarkable.
Violin plot	A box plot with the density drawn around it: shape and summary at once	Reads badly with small samples.

Use all three, not one. The histogram gives shape, the boxplot gives outliers and the summary, and together they answer every question you need for imputation and scaling. The single most useful habit in EDA is drawing a histogram and a boxplot of the same column side by side.

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

sns.histplot(df["Age"],  kde=True, bins=30, ax=axes[0, 0])
axes[0, 0].set_title("Age: histogram + KDE")

sns.boxplot(x=df["Age"], ax=axes[0, 1])
axes[0, 1].set_title("Age: boxplot")

sns.histplot(df["Fare"], kde=True, bins=40, ax=axes[1, 0])
axes[1, 0].set_title("Fare: histogram + KDE")

sns.boxplot(x=df["Fare"], ax=axes[1, 1])
axes[1, 1].set_title("Fare: boxplot")

plt.tight_layout(); plt.show()

# The numbers behind the pictures
df[["Age", "Fare"]].describe()
df[["Age", "Fare"]].skew()
df[["Age", "Fare"]].kurt()      # Age 0.18, Fare 33.40 -- a very heavy tail
```

What the Titanic numeric columns actually look like:

- **`Age`**: one broad hump centred near 28, a distinct bulge of children under 10, a thin tail to 80. Skew 0.39, kurtosis 0.18: close to normal. Eleven points beyond the upper fence, all genuinely elderly. *Decision: median imputation, standard scaling, and consider binning to give the model the child effect directly.*
- **`Fare`**: a wall at the left edge, 50% of passengers below 14.45, 75% below 31.00, and a tail running to 512.33. Skew 4.79, kurtosis 33.40. Fifteen passengers paid nothing. *Decision: median, a log transform or robust scaling, and `FarePerPerson` to address the cause.*
- **`SibSp` and `Parch`**: dominated by zero, 537 passengers travelled with neither sibling, spouse, parent nor child. Skew 3.70 and 2.75. *Decision: not really continuous. Combine into*

`FamilySize` and derive `IsAlone`.

7.2 Categorical variables

Chart	What it is for
Count plot	Frequency of each category, straight from the raw column. `sns.countplot` does the tallying for you.
Bar chart	Any aggregate per category, not just counts: mean survival, median fare. `sns.barplot` plus an estimator.
Pie chart	Proportions of a whole, for three or four categories at most. Humans compare lengths well and angles badly; a bar chart is almost always better.
`value_counts`	Not a chart, and often the fastest answer. Use `normalize=True` for proportions and `dropna=False` to see the gaps.

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, col in zip(axes, ["Sex", "Pclass", "Embarked"]):
    sns.countplot(x=col, data=df, ax=ax)
    ax.set_title(col)
plt.tight_layout(); plt.show()

for col in ["Sex", "Pclass", "Embarked", "Survived"]:
    print(df[col].value_counts(dropna=False), "\n")
    print(df[col].value_counts(normalize=True).round(4), "\n")
```

Column	Counts	What it means for modelling
`Survived`	549 died, 342 survived (38.38%)	Mild imbalance. Accuracy is misleading enough here that a constant predictor scores 61.62%. Stratify the split; report precision, recall and ROC-AUC.
`Sex`	577 male, 314 female	Binary, no missing values. Map to 0/1.
`Pclass`	491 third, 216 first, 184 second	Ordinal. Third class dominates, which will matter for every class-conditional statistic.
`Embarked`	644 S, 168 C, 77 Q, 2 missing	`Q` has only 77 rows, so a one-hot column for it will be sparse and its coefficient unstable.

The class balance is a univariate finding with the largest consequences. 38.38% positive tells you the baseline to beat is 61.62%, not 50%. It tells you to pass `stratify=y` to the split. It tells you accuracy alone is not a sufficient report. All of that comes from one `value_counts`.

Common mistakes

- **Accepting the default bin count.** `sns.histplot` picks bins heuristically. Try 20, 30 and 50 before you believe a shape.

- **Plotting the target as though it were a feature.** `Survived` gets a `value_counts`, not a histogram.
- **Ignoring the tiny categories.** 77 rows for `Q` is fine here and would be a problem in a 20-level column, where rare levels need grouping into `Other`.
- **Skipping the boxplot because the histogram looked fine.** The histogram of a skewed variable compresses the tail into invisibility.
- **Reading a KDE as though it were data.** It is a smoothed estimate and it will happily show density where no observation exists.

Industry best practices

- **Loop over the columns.** Write one loop that plots every numeric column and one that plots every categorical column. Consistency beats bespoke charts.
- **Write a one-line conclusion under every plot.** If you cannot, the plot was not worth drawing.
- **Check cardinality before plotting a categorical.** A countplot of 681 tickets is a solid black rectangle.
- **Automate the first pass** with a profiling tool, then do the thinking by hand. The tool finds the columns; it does not make the decisions.

Interview questions

1. **What is univariate analysis and what is it for?**
   Examining one variable at a time to establish centre, spread, shape, extremes and gaps, which is what imputation, scaling and encoding decisions are made from.
2. **Histogram or box plot?**
   Both. The histogram gives shape and modality, the box plot gives the rank-based summary and the outliers. Neither is sufficient alone.
3. **What does the bin count change?**
   The apparent shape. Too few bins hide a second mode, too many turn sampling noise into structure.
4. **Your target is 38% positive. What follows from that?**
   Baseline accuracy is 62%, so report precision, recall, F1 and ROC-AUC, and stratify the split.
5. **When is a pie chart acceptable?**
   Three or four categories that make up a whole, where the reader needs proportions rather than comparisons. A bar chart is usually clearer.

8. Bivariate Analysis

Definition: Analysis of two variables together, to see whether and how they move with one another. In supervised learning one of the two is usually the target, and this is where feature selection begins.

Goal: Find the relationships worth modelling, and the ones that are artefacts. Bivariate analysis produces the sentences that end up in the report: *women survived at 74% and men at 19%.*

8.1 Numerical versus numerical: the scatter plot

```
sns.scatterplot(x="Age", y="Fare", data=df, hue="Survived", alpha=0.6)
plt.title("Age vs Fare, coloured by survival")
plt.show()

df[["Age", "Fare"]].corr()          # Pearson r = 0.096
```

What Age against Fare shows: almost nothing, and that is a result. The correlation is 0.096, effectively zero. Points are dense in the lower left, ages 20 to 40 at fares under 50, with a scattering of high fares across all ages. Older passengers did not systematically pay more.

The lesson in a null result: a scatter plot that shows no relationship has saved you from building a feature. It also demonstrates that a correlation near zero means no *linear* relationship, not no relationship: the same plot coloured by `Survived` does show structure, because the high-fare points are disproportionately orange.

Pattern in a scatter	Interpretation
Upward band	Positive relationship. Tight band means strong.
Downward band	Negative relationship.
Formless cloud	No linear relationship. Check for a curved one before concluding none.
Curve or fan	Non-linear. Consider a transform, or a model that does not assume linearity.
Distinct clumps	A hidden categorical variable. Colour by candidates until the clumps separate.
Horizontal or vertical stripes	Rounded, binned or imputed values. Stripes at one exact value usually mean imputation.

Overplotting: with 891 points a scatter is readable. With 100,000 it is a solid block, and `alpha=0.1`, a sample, a hexbin or a 2-D density plot is the fix.

8.2 Categorical versus numerical: the box plot

```
sns.boxplot(x="Pclass", y="Fare", data=df)
plt.title("Fare by passenger class")
plt.show()

df.groupby("Pclass")["Fare"].agg(["median", "mean", "count"])
#          median     mean  count
# 1        60.2875  84.1547   216
# 2        14.2500  20.6622   184
```

```
# 3         8.0500  13.6756   491
```

What Pclass against Fare shows: exactly what it should, which is the point. Median fares are 60.29, 14.25 and 8.05 for first, second and third class. The two variables encode overlapping information; the correlation between them is −0.549, the strongest in the dataset. A linear model given both will split the credit between them and neither coefficient will be interpretable.

The boxes also locate the outliers. 104 of the 116 high-fare outliers sit inside the first-class box. Within its own class, a fare of 200 is far less remarkable than it looked in the full-column boxplot. This is why the class-conditional view matters: outliers are relative to the comparison group you choose.

8.3 Categorical versus the target

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col in zip(axes, ["Sex", "Pclass", "Embarked"]):
    sns.countplot(x=col, hue="Survived", data=df, ax=ax)
    ax.set_title(f"Survival by {col}")
plt.tight_layout(); plt.show()

# Counts answer "how many", rates answer "how likely". You want rates.
df.groupby("Sex")["Survived"].agg(["mean", "count"])
pd.crosstab(df["Sex"], df["Survived"], normalize="index").round(4)

# Rates with confidence intervals, in one call
sns.barplot(x="Pclass", y="Survived", hue="Sex", data=df)
```

Group	n	Survived	Reading
Female	314	74.20%	The strongest single predictor in the dataset.
Male	577	18.89%	A 55-point gap: “women and children first” is in the data.
1st class	216	62.96%	
2nd class	184	47.28%	Monotonic in class, which is why `Pclass` should be treated as ordinal.
3rd class	491	24.24%	
Embarked C	168	55.36%	Not about the port: Cherbourg boarded a disproportionate number of first-class passengers.
Embarked Q	77	38.96%	
Embarked S	644	33.70%	

Counts versus rates, the mistake worth naming: a countplot of `Pclass` by `Survived` shows the tallest survivor bar in *first* class and the tallest death bar in third. But third class has 491 passengers and first has 216. Raw counts conflate group size with group risk. Always convert to a rate before drawing a conclusion.

Confounding, on display: embarking at Cherbourg appears to raise survival by 22 points over Southampton. It does not. Cherbourg boarded a much higher proportion of first-class passengers, and class is the real driver. This is why bivariate analysis is never the last word: you need the third variable to see it, which is the next section.

Common mistakes

- **Reading counts as rates.** The single most common misreading of a countplot.

- **Concluding causation.** Cherbourg does not cause survival.
- **Stopping at Pearson's r.** It measures linear association only. A perfect U-shaped relationship has r near zero.
- **Ignoring group sizes.** A 100% survival rate over 3 passengers is not a finding.
- **Testing every pair.** With enough comparisons some will look significant by chance. Start from hypotheses, not from the full grid.

Industry best practices

- **Always report n beside the rate.** A rate without a denominator is not a result.
- **Prefer the rate plot** (`barplot` with the target as y) over the count plot when the question is about likelihood.
- **Check the obvious confounder** before writing down any bivariate finding. Class explains most of the Titanic ones.
- **Keep an effect-size ranking** of features against the target. It is your feature-selection shortlist and your sanity check on the model's importances later.

Interview questions

1. **Which plot for numerical versus numerical, categorical versus numerical, and categorical versus categorical?**
   Scatter; box or violin; countplot with hue or a normalised crosstab.
2. **Correlation is 0.02. Is there no relationship?**
   There is no linear *relationship. Plot it; a U-shape gives r near zero.*
3. **Cherbourg passengers survived more. Does the port matter?**
   Probably not. Cherbourg boarded more first-class passengers, and class is the driver. Control for it before concluding.
4. **Why is a countplot misleading for comparing survival across classes?**
   Bar height mixes group size with group risk. Use rates.
5. **How would you quantify the association between two categorical variables?**
   A chi-squared test of independence, and Cramér's V for the effect size.

9. Multivariate Analysis

Definition: Analysis of three or more variables at once. It exists because bivariate analysis lies by omission: two variables can appear related purely because a third drives both.

Goal: Find confounders, find redundancy between features, and find interactions, places where the effect of one variable depends on the value of another.

9.1 The pair plot

```
sns.pairplot(
    df[["Age", "Fare", "SibSp", "Parch", "Survived"]],
    hue="Survived", diag_kind="hist", corner=True, plot_kws={"alpha": 0.5},
)
plt.show()
```

A pair plot draws every numeric column against every other in a grid, with each variable's own distribution down the diagonal.

Pros	Cons
Every pairwise relationship in one object	Grows as the square of the column count: 10 columns is 100 panels
`hue` adds the target to every panel at once	Unreadable above roughly 6 or 7 variables
Shows non-linear shapes a correlation number cannot	Slow on large data; sample before plotting
The diagonal gives you the univariate view for free	Numeric columns only; ignores categoricals entirely

When to use it: early, on a shortlist of 4 to 6 numeric candidates, as a fast scan. Use `corner=True` to draw only the lower triangle, since the upper is a mirror, and sample the rows above about 5,000.

When not to: wide datasets, mostly-categorical datasets, and as a substitute for thinking about which pairs matter.

9.2 The correlation matrix and heatmap

Concept: Pearson's correlation coefficient measures the strength and direction of the *linear* relationship between two numeric variables. It runs from -1 to $+1$ and is scale-free, so it can be compared across pairs of variables measured in different units.

$$r_{xy} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2}\,\sqrt{\sum(y_i - \bar{y})^2}}$$

```
numeric = df[["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]]

plt.figure(figsize=(8, 6))
sns.heatmap(numeric.corr(), annot=True, fmt=".2f",
            cmap="coolwarm", center=0, vmin=-1, vmax=1, square=True)
plt.title("Correlation matrix")
plt.show()
```

```
# Just the target column, ranked. Usually the only column you need.
numeric.corr()["Survived"].drop("Survived").sort_values(ascending=False)
```

	Survived	Pclass	Age	SibSp	Parch	Fare
Survived	1.000	−0.338	−0.077	−0.035	0.082	0.257
Pclass	−0.338	1.000	−0.369	0.083	0.018	−0.549
Age	−0.077	−0.369	1.000	−0.308	−0.189	0.096
SibSp	−0.035	0.083	−0.308	1.000	0.415	0.160
Parch	0.082	0.018	−0.189	0.415	1.000	0.216
Fare	0.257	−0.549	0.096	0.160	0.216	1.000

$\|r\|$	Strength	Titanic instance
0.00 to 0.19	Very weak or none	`Age` vs `Survived` (−0.077): no linear effect, though children clearly did better, which is the non-linearity r cannot see.
0.20 to 0.39	Weak to moderate	`Fare` vs `Survived` (+0.257): richer passengers survived more. `Pclass` vs `Survived` (−0.338): negative because class 3 is the *lowest* class, so the number goes up as status goes down.
0.40 to 0.59	Moderate	`SibSp` vs `Parch` (+0.415): families travel together. `Pclass` vs `Fare` (−0.549): the strongest pair here.
0.60 to 0.79	Strong	None in this dataset.
0.80 to 1.00	Very strong; suspect redundancy or leakage	None here. If you see 0.95 between two features, one of them is probably a copy of the other.

Reading the sign correctly matters more than reading the size. `Pclass` and `Survived` correlate at −0.338. That is not "class does not help". It is "as the class *number* increases from 1 to 3, survival falls", which is a strong positive effect of status stated in the encoding's own direction. Whenever an ordinal is coded with the best category as the lowest number, every correlation involving it reads backwards.

9.3 The four correlation traps

1. Correlation is not causation. `Fare` correlates with survival at 0.257. Paying more did not buy a lifeboat seat; it bought a cabin on an upper deck, closer to the boats. The fare is a proxy.

2. Pearson sees only straight lines. `Age` against `Survived` is −0.077, essentially nothing. Bin the same column and the effect appears at once: children aged 0–12 survived at 57.97%, everyone between 12 and 60 at roughly 38–40%, and passengers over 60 at 22.73%. The relationship is not a line, it is a plateau with a step up at one end and down at the other, and a linear coefficient averages it away. This is the strongest single argument for binning `Age`.

```
# Spearman ranks instead of values, so it catches any monotonic relationship
df["Fare"].corr(df["Survived"], method="spearman")   # 0.324 vs Pearson's 0.257

# The relationship Pearson missed entirely
df.groupby(pd.cut(df["Age"], [0, 12, 20, 40, 60, 80]))["Survived"].mean()
```

3. `df.corr()` silently ignores your categoricals. `Sex` is the single strongest predictor in this dataset, a 55-point survival gap, and it does not appear in the correlation matrix at all because it is not numeric. A heatmap is a summary of the numeric columns, not of the dataset. For a binary against a numeric use the point-biserial correlation; for two categoricals use chi-squared and Cramér's V.

4. Correlated features are not the same as useful features. Two features correlated at 0.9 with each other carry one feature's worth of information and destabilise any linear model's coefficients. This is multicollinearity, and it is a property of the feature set, not of any one feature.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = df[["Pclass", "Age", "SibSp", "Parch", "Fare"]].dropna()
vif = pd.DataFrame({
    "feature": X.columns,
    "VIF": [variance_inflation_factor(X.values, i) for i in range(X.shape[1])],
})
# VIF above 5 is worth a look, above 10 is a problem.
```

What to do about multicollinearity: drop one of the pair, combine them into one feature (`SibSp` and `Parch` become `FamilySize`), or use a model that does not care. Trees are unbothered; ridge and lasso regularise their way through it; plain linear and logistic regression produce unstable and uninterpretable coefficients.

9.4 Three variables at once

```
# The confounder from Section 8, resolved
sns.barplot(x="Pclass", y="Survived", hue="Sex", data=df)

pd.crosstab([df["Sex"], df["Pclass"]], df["Survived"], normalize="index").round(4)

# Small multiples: one panel per level of a third variable
g = sns.FacetGrid(df, col="Pclass", row="Sex", height=3)
g.map(sns.histplot, "Age")
```

Survival rate	1st class	2nd class	3rd class
Female	96.81%	92.11%	50.00%
Male	36.89%	15.74%	13.54%

This one table is the whole dataset. Sex dominates: a third-class woman (50.00%) outlived a first-class man (36.89%). Class matters, but far more for women, whose rate falls 47 points from first to third, than for men, whose rate falls 23. That is an *interaction*: the effect of class depends on sex. A linear model given `Pclass` and `Sex` as separate terms cannot represent it, which is why a tree-based model tends to do slightly better here, and why an explicit `Sex * Pclass` interaction term helps a logistic regression.

Common mistakes

- **Treating the heatmap as feature selection.** It ranks linear association with the target among numeric columns only. `Sex` is invisible to it.
- **Dropping a feature for low correlation.** Low r means low *linear* association. `Age` at -0.077 is genuinely useful once binned.
- **Correlating an ordinal or an ID.** `Pclass` is defensible with care; `PassengerId` is not.
- **Annotating a 40 by 40 heatmap.** Nobody can read it. Filter to the target column, or mask everything below a threshold.
- **Ignoring the sign convention of an ordinal.** A negative correlation with `Pclass` usually means a positive effect of status.

Industry best practices

- **Report Spearman alongside Pearson** for skewed data. On `Fare` versus `Survived` it is 0.324 against 0.257, and the rank version is the more honest summary.
- **Check VIF** before interpreting any linear model's coefficients.
- **Use the target correlation column**, sorted, rather than the full matrix. One column, ranked, answers the actual question.
- **Look for interactions deliberately** on the two or three strongest features. They are where the remaining accuracy usually is.
- **Mask the upper triangle** with `np.triu` so the heatmap shows each pair once.

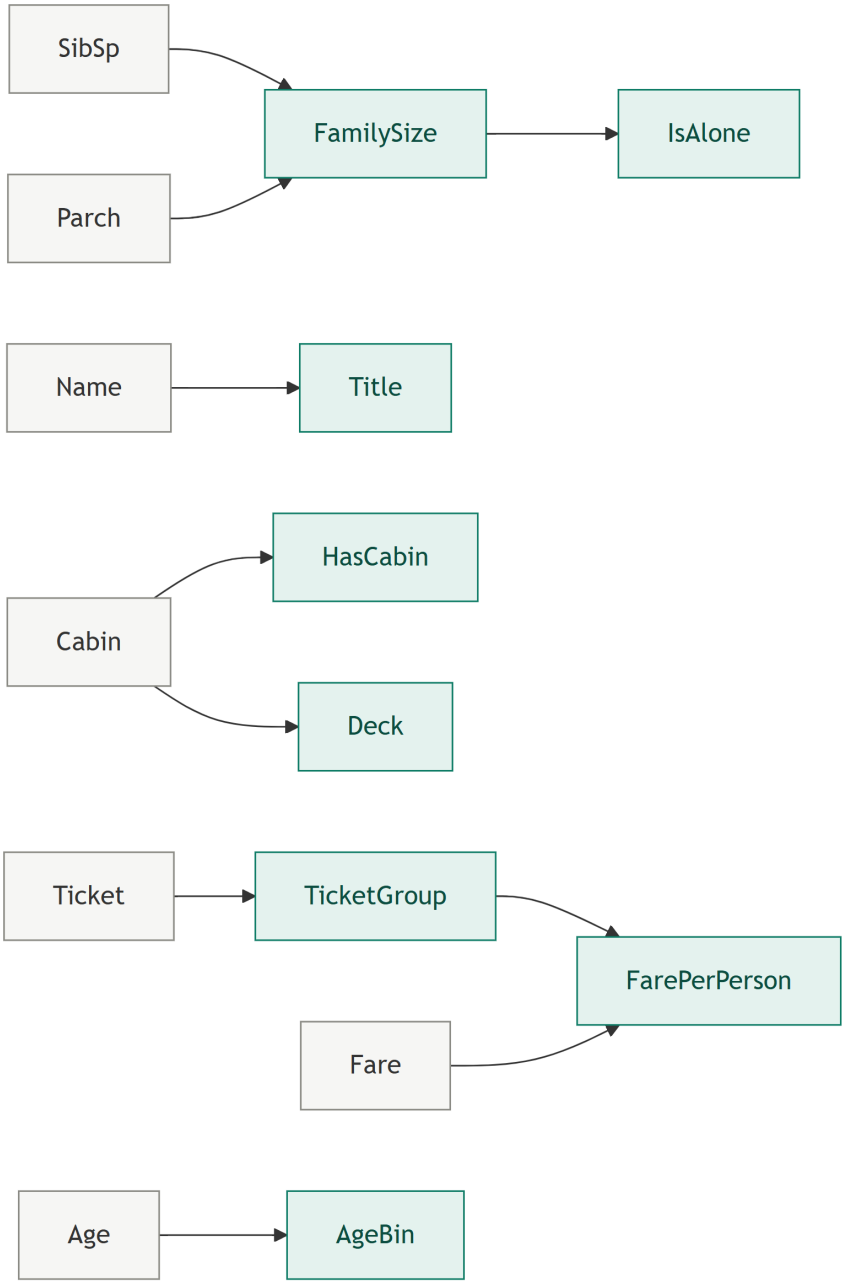
Interview questions

1. **What does a correlation of -0.55 between `Pclass` and `Fare` mean?**
   A moderate negative linear relationship: as the class number rises from 1 to 3, fares fall. Since class 1 is the highest class, higher status means higher fare.
2. **Why can a feature with near-zero correlation still be predictive?**
   Pearson measures only linear association. `Age` at -0.077 hides a U-shape: children survived, young adults did not.
3. **Pearson versus Spearman?**
   Pearson on raw values, catching linear relationships and sensitive to outliers; Spearman on ranks, catching any monotonic relationship and robust.
4. **What is multicollinearity, how do you detect it, and what do you do?**
   Predictors correlated with each other, which destabilises coefficients. Detect with a correlation matrix or VIF above 5 to 10. Drop, combine, regularise, or use a tree.
5. **Why does `df.corr()` omit `Sex`?**
   It is an object column and Pearson needs numerics. Encode it first, or use point-biserial or Cramér's V.
6. **What is an interaction effect, with an example from this data?**
   When the effect of one variable depends on another. Class costs women 47 points from first to third and men only 23.

10. Feature Engineering

Short description: Creating new columns from existing ones so that the pattern you want the model to learn becomes easy for it to see. It is the highest-leverage work in applied machine learning and the part that does not generalise: it requires knowing what the data means.

Why engineered features improve models: every algorithm has a hypothesis space, a set of functions it can express. Logistic regression can express a weighted sum. It cannot express "travelling with 2 to 4 relatives was safer than travelling alone or in a large group", because that is not monotonic in `SibSp` or `Parch`. Feature engineering moves the pattern out of the space the model cannot reach and into the space it can. A tree could learn `FamilySize` eventually, given enough depth and enough rows; handing it the column directly costs one line and saves both.



10.1 FamilySize and IsAlone

Concept: `SibSp` counts siblings and spouses, `Parch` counts parents and children. Neither alone describes the group a passenger was travelling in. Their sum plus one does.

```
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"]    = (df["FamilySize"] == 1).astype(int)

df.groupby("FamilySize")["Survived"].agg(["mean", "count"])
df.groupby("IsAlone")["Survived"].agg(["mean", "count"])
```

FamilySize	1	2	3	4	5	6	7	8+
Survived	30.35%	55.28%	57.84%	72.41%	20.00%	13.64%	33.33%	0.00%
n	537	161	102	29	15	22	12	13

This is exactly the shape a linear model cannot represent. Survival climbs from 30% alone to 72% in a family of four, then collapses to zero for families of eight or more. Correlations of `SibSp` and `Parch` with `Survived` are −0.035 and 0.082, both effectively nothing, because a monotonic summary of a hump is zero. Give the model `FamilySize` plus a binned version, or `IsAlone`, and the pattern is available.

`IsAlone` in one number: alone, 30.35% over 537 passengers; with anybody at all, 50.56% over 354. A binary column recovering a 20-point gap from two columns that individually showed nothing.

10.2 Title extraction from Name

Concept: `Name` is 891 unique strings, useless as a category. But every name in this manifest follows the pattern `Surname, Title. Given names`, and the title encodes sex, marital status, approximate age and social rank in one token.

```
df["Title"] = df["Name"].str.extract(r",\s*([^\.]+)\.")[0].str.strip()

df["Title"].value_counts()
# Mr 517, Miss 182, Mrs 125, Master 40, Dr 7, Rev 6, Major 2, Mlle 2,
# Col 2, Don 1, Mme 1, Ms 1, Lady 1, Sir 1, Capt 1, the Countess 1, Jonkheer 1

# Normalise the equivalents, then pool everything rare
df["Title"] = df["Title"].replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"})
rare = df["Title"].value_counts()[lambda s: s < 10].index
df["Title"] = df["Title"].where(~df["Title"].isin(rare), "Rare")
# Mr 517, Miss 185, Mrs 126, Master 40, Rare 23
```

Title	n	Survived	Median age	What it encodes
Mr	517	15.67%	30.0	Adult male
Mrs	126	79.37%	35.0	Married adult female
Miss	185	70.27%	21.0	Unmarried female, skewing younger
Master	40	57.50%	3.5	**Boy under about 13.** This is the one that pays.
Rare	23	34.78%	48.5	Titled, military and clerical passengers

Why `Master` matters: `Sex` splits the data into 18.89% and 74.20%, and every male is on the low side. `Master` pulls 40 boys out of that group at 57.50%, nearly the female rate. No combination of `Sex` and `Age` gives the model that group cleanly, because `Age` is missing for 177 rows and the cutoff is not a round number. A regular expression over a text column recovers it.

The second payoff: `Title` is the best available predictor of the missing ages. Median age is 3.5 for `Master`, 21 for `Miss`, 30 for `Mr`, 35 for `Mrs`. Grouping by `Title` and `Pclass` for imputation, as in Section 4, is only possible because this feature exists.

10.3 Everything else worth building here

Feature	Definition	Payoff
`HasCabin`	`Cabin.notna()`	66.67% survival against 29.99%. Recovers the signal from a column you are about to drop.
`Deck`	First letter of `Cabin`	147 cabins collapse to 8 decks: C 59, B 47, D 33, E 32, A 15, F 13, G 4, T 1. Physical location on the ship.
`TicketGroup`	Count of rows sharing the ticket	Reveals travelling groups that `SibSp` and `Parch` miss: nannies, friends, servants.
`FarePerPerson`	`Fare / TicketGroup`	Fixes the fare outliers at the source. The 512.33 ticket becomes 170.78 and flagged outliers fall from 116 to 59.
`AgeBin`	`pd.cut` into child / teen / adult / senior	Makes the non-linear age effect visible to a linear model.
`FareBin`	`pd.qcut` into quartiles	Equal-sized groups, which is robust to the skew without a transform.

```python
df["HasCabin"]      = df["Cabin"].notna().astype(int)
df["Deck"]          = df["Cabin"].str[0].fillna("Unknown")
df["TicketGroup"]   = df["Ticket"].map(df["Ticket"].value_counts())
df["FarePerPerson"] = df["Fare"] / df["TicketGroup"]

# pd.cut: fixed edges you choose, meaningful labels, unequal group sizes
df["AgeBin"] = pd.cut(df["Age"], bins=[0, 12, 20, 40, 60, 100],
                      labels=["child", "teen", "adult", "middle", "senior"])

# pd.qcut: equal-sized groups, edges chosen by the data
df["FareBin"] = pd.qcut(df["Fare"], q=4, labels=["low", "mid", "high", "top"])
```

`cut` versus `qcut`: `cut` splits on values you choose, giving groups that mean something (a child is under 12) and group sizes you do not control. `qcut` splits on quantiles, giving equal group sizes and boundaries that mean nothing in particular. Use `cut` when the domain supplies the boundaries and `qcut` when it does not, which for `Fare` it does not.

The cost of binning: you throw information away. A 39-year-old and a 21-year-old become the same value. Bin when the relationship is genuinely non-linear and the model is linear; do not bin by default, and never bin for a tree, which finds its own thresholds.

10.4 Feature engineering can leak

The rule: a feature built from a statistic of the whole dataset is a leak, because the statistic includes the test rows.

Feature	Safe?	Why
`SibSp + Parch + 1`	Safe	Row-wise arithmetic. Depends on nothing outside the row.
Title from `Name`	Safe	A regular expression on one cell.
`Fare > df["Fare"].median()`	**Leaks**	The median is computed over every row, test rows included. Fit it on train and store it.
`qcut` on the full column	**Leaks**	The bin edges are quantiles of the whole dataset.
Target / mean encoding	**Leaks badly**	Uses the target directly. Needs out-of-fold computation or smoothing, or it memorises the answer.
`TicketGroup` count	**Borderline**	Counts rows across the split. Defensible if the group is known at prediction time; state the assumption.

The test that catches all of them: could you compute this feature for a single new passenger, arriving alone, knowing nothing about the rest of the dataset? If yes, it is safe. If it needs a population statistic, that statistic has to be learned on the training fold and stored, exactly like a scaler's mean.

Common mistakes

- **Building features from full-dataset statistics.** The commonest leak, and the hardest to see in a notebook.
- **Target-encoding without out-of-fold computation.** Near-perfect training accuracy, useless model.
- **Engineering before understanding.** `FamilySize` is obvious only once you have seen that `SibSp` and `Parch` each correlate with nothing.
- **Keeping the raw column and the derived one** without checking. `SibSp`, `Parch` and `FamilySize` together are collinear by construction.
- **Binning everything.** It discards resolution and helps only where the relationship is genuinely non-linear and the model is linear.
- **Not validating the gain.** A feature that does not move cross-validated score is complexity you now have to maintain.

Industry best practices

- **One function per feature**, named after what it computes, so the same code runs in training and in serving. Divergence between the two is the classic production bug.
- **Measure each feature's contribution** with cross-validated score deltas, not intuition.
- **Ask the domain expert.** `Master` meaning "boy" is not a fact you derive from the data; it is a fact you know or ask about.
- **Version the feature definitions** with the model. A model artefact that does not pin its feature code cannot be reproduced.

- **Prefer few strong features** to many weak ones. Every column is surface area for drift.

Interview questions

1. **What is feature engineering and why does it help?**
   Creating columns that expose a pattern the model's hypothesis space cannot otherwise express. `FamilySize` *turns a non-monotonic relationship in two columns into one the model can use.*

2. **Give three engineered features for Titanic and justify each.**
   `FamilySize`*, because survival peaks at four and neither source column shows it;* `Title`*, because* `Master` *isolates 40 boys at 57.5% survival;* `HasCabin`*, because the missingness carries a 37-point gap.*

3. **How can feature engineering cause leakage?**
   Any feature built from a statistic over the full dataset, quantile edges, means, target encodings, encodes test-set information.

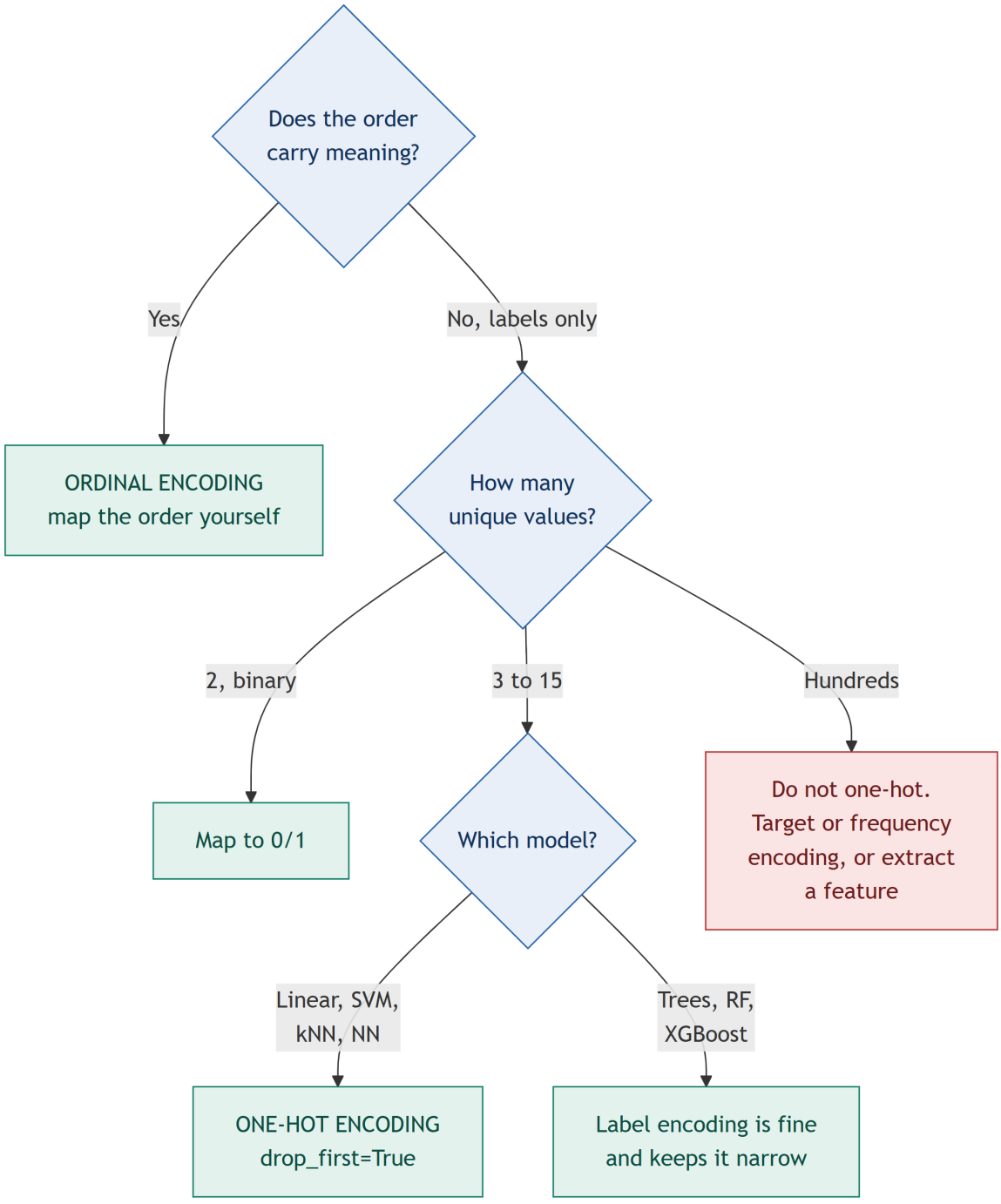
4. **When should you bin a continuous variable?**
   When the relationship is non-linear and the model is linear, or when the domain has real boundaries. Not for tree models.

5. **`cut` or `qcut`?**
   `cut` *for meaningful fixed edges and uneven groups;* `qcut` *for equal-sized groups and data-driven edges.*

6. **How do you know a new feature actually helped?**
   Cross-validated score with and without it, on the same folds and seed.

11. Encoding Categorical Variables

Why models require numbers: every estimator is arithmetic underneath. Logistic regression computes $w^\top x + b$. A decision tree compares a value to a threshold. A neural network multiplies matrices. None of those operations are defined on the string `"female"`. Encoding is the translation, and the only question is which translation tells the model the truth.



11.1 Label encoding

Concept: Assign each category an integer, usually in alphabetical order. One column in, one column out.

```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["Sex_encoded"] = le.fit_transform(df["Sex"])    # female -> 0, male -> 1
le.classes_                                        # array(['female', 'male'])

# The pandas equivalent, and the one you should prefer for a binary
df["Sex_encoded"] = (df["Sex"] == "male").astype(int)
```

Advantages	Disadvantages
One column per variable: no dimensionality growth	Invents an order that may not exist
Trivial to compute and to invert	Invents distances: 2 minus 0 equals 2, so C and S are "twice as far apart" as C and Q
Memory-efficient on high-cardinality columns	The order is alphabetical, so it is arbitrary
Works well with tree-based models, which only compare	`LabelEncoder` takes a 1-D input, so it fits one column at a time and cannot go straight into a `ColumnTransformer`

When to use it: binary variables, where the false ordering is harmless because there is only one gap; tree-based models, which split on thresholds and never on distance; and the target column of a classification problem, which is what `LabelEncoder` is actually designed for.

When NOT to use it: nominal variables with three or more levels feeding a linear, distance-based or neural model. Encoding `Embarked` as C=0, Q=1, S=2 tells a logistic regression that Queenstown lies exactly halfway between Cherbourg and Southampton, and the model fits a coefficient to that fiction.

Ordinal encoding is the version that is usually meant. `OrdinalEncoder` lets you supply the order rather than accepting the alphabet. For `Pclass` the order is real, so this preserves information instead of inventing it:

```python
from sklearn.preprocessing import OrdinalEncoder

enc = OrdinalEncoder(categories=[["Low", "Medium", "High"]])
# Pclass already carries its order as 1, 2, 3 -- no encoding needed at all.
# Reverse it if you want the number to rise with status:
df["Pclass_rank"] = 4 - df["Pclass"]     # 3 = first class, 1 = third
```

11.2 One-hot encoding

Concept: Replace one categorical column with one binary column per category. A passenger who boarded at Cherbourg gets 1 in `Embarked_C` and 0 in the others. No ordering is implied because there is no single axis left to order along.

```python
# pandas: fast, fine for exploration
pd.get_dummies(df, columns=["Sex", "Embarked"], drop_first=True, dtype=int)

# sklearn: the one that goes into production
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore")
ohe.fit_transform(X_train[["Embarked"]])   # learns the categories here
ohe.transform(X_test[["Embarked"]])        # reuses exactly those categories
```

```
ohe.get_feature_names_out()              # ['Embarked_Q', 'Embarked_S']
```

Advantages	Disadvantages
No false ordering and no false distances	One column per category: 50 categories becomes 50 columns
Every category gets its own coefficient, which is directly interpretable	Sparse, high-dimensional matrices slow down and destabilise distance-based models
The standard for linear models, SVMs and neural networks	A rare category produces a nearly-constant column and an unstable coefficient
`handle_unknown="ignore"` survives an unseen category at prediction time	A category unseen in training has no column at all, so `get_dummies` silently produces a different matrix shape

The dummy variable trap, and `drop_first`: three ports produce three columns whose values always sum to 1, so any one of them is exactly determined by the other two. That perfect collinearity makes the ordinary least squares solution non-unique and the coefficients meaningless. Dropping one column, `drop_first=True`, removes the redundancy; the dropped category becomes the baseline that every other coefficient is measured against. Drop it for linear models. Keep every column for tree models and for regularised regressions, where the redundancy is harmless and dropping one makes the importances harder to read.

The production trap in `get_dummies`: it creates columns from whatever values it happens to see. Call it on the training set and you get `Embarked_Q` and `Embarked_S`. Call it on a test batch where nobody boarded at Queenstown and you get one column. The model then receives a matrix of the wrong width, and either raises or, worse, silently misaligns the features. `OneHotEncoder` stores the categories at fit time and always emits the same columns. Use `get_dummies` in a notebook and `OneHotEncoder` in anything that runs twice.

11.3 The Titanic decisions

Column	Levels	Choice	Reason
`Sex`	2	Binary map to 0/1	With two levels, label and one-hot are the same column. No reason to add a second.
`Embarked`	3	One-hot, `drop_first`	Nominal. C, Q and S have no order to preserve. Two columns after dropping the baseline.
`Pclass`	3	Leave as 1, 2, 3 (ordinal)	The order is real and monotonic in survival: 62.96%, 47.28%, 24.24%. One-hot would discard that.
`Title`	5	One-hot	Nominal after pooling rare titles. Five columns, four after dropping the baseline.
`Deck`	9	One-hot, or group first	Nominal, and `T` has one passenger. Pool the small decks before encoding.
`Ticket`	681	Never one-hot	681 columns of almost pure noise. Use `TicketGroup` instead.

Measured, not assumed: five-fold cross-validated logistic regression on this feature set scores **0.7935** with `Embarked` one-hot encoded and **0.7913** with it label-encoded. The gap is small because `Embarked` is a weak feature with only three levels and one dominant category. The principle still holds, and on a ten-level nominal it would not be small: the point of one-hot encoding is that its cost is bounded and its correctness does not depend on how lucky the alphabetical order was.

11.4 The other encoders, and when they are worth it

Method	How it works	When
Frequency / count	Replace each category with how often it occurs	High cardinality, tree models. One column, no ordering claim beyond frequency.
Target / mean	Replace each category with the mean target for that category	High cardinality with a real signal. **Leaks badly** unless computed out-of-fold and smoothed toward the global mean.
Binary encoding	Encode the label index in base 2, one column per bit	Dozens to hundreds of categories, where one-hot is too wide.
Hashing	Hash the category into a fixed number of columns	Very high or unbounded cardinality, streaming data. Collisions are accepted in exchange for a fixed width.
Embeddings	Learn a dense vector per category	Neural networks with many categories and enough data to train the table.
Leave as-is	LightGBM and CatBoost accept categorical columns natively	When you are using them. Usually better than anything you would do by hand.

Cardinality is the deciding variable. Two levels: map to 0/1. Three to about fifteen: one-hot. Fifteen to a few hundred: group the rare levels into `Other` first, then one-hot, or use target encoding with proper out-of-fold computation. Thousands: hashing, embeddings, or extract a feature instead, which is exactly what `Title` does to `Name`.

Common mistakes

- **Label-encoding a nominal for a linear model.** The defining mistake of this section.
- **`get_dummies` on train and test separately.** Different columns, misaligned matrices, silent wrongness.
- **Encoding before splitting.** The encoder's category list is a learned parameter like any other.
- **One-hot encoding a high-cardinality column.** 681 ticket columns is not a feature set.
- **Forgetting `handle_unknown`.** A category that appears for the first time in production raises, and the service goes down at 3am.
- **Target encoding without out-of-fold folds.** Perfect training score, no model.
- **Destroying a real order.** One-hot on `Pclass` discards that first beats third.

Industry best practices

- **Always `OneHotEncoder` inside a `ColumnTransformer`**, never `get_dummies`, for anything that will run more than once.
- **Set `handle_unknown="ignore"`** and decide deliberately what an unseen category should mean.
- **Group rare categories** into `Other` using a training-set frequency threshold, and store the threshold.
- **Keep `get_feature_names_out()`** with the model. Without it, a coefficient vector is a list of anonymous numbers.
- **Monitor category drift.** New levels appearing in production is a data contract change, not a modelling problem.

Interview questions

1. **Label encoding versus one-hot: when do you use each?**
   Label for binary columns, ordinals with a real order, and tree models. One-hot for nominal columns of low to moderate cardinality feeding linear, distance-based or neural models.
2. **What is the dummy variable trap?**
   One-hot columns sum to 1, so they are perfectly collinear and the OLS solution is not unique. `drop_first=True` *removes the redundancy.*
3. **Why is one-hot bad for a 500-category column?**
   500 sparse columns: memory, dilution of distance metrics, unstable coefficients. Use target, binary or hashing encoding instead.
4. **`get_dummies` or `OneHotEncoder`?**
   `OneHotEncoder` *for anything reproducible: it remembers its categories and handles unseen ones.* `get_dummies` *infers columns from the data it is given each time.*
5. **What is target encoding and what is its danger?**
   Replacing a category with its mean target. It uses the label, so without out-of-fold computation and smoothing it memorises the training answers.
6. **How would you encode `Pclass`, and why?**
   Leave it as 1, 2, 3. It is ordinal, the order is real, and survival is monotonic in it.

12. Feature Scaling

Short description: Putting numeric features onto a comparable range so that no feature dominates a calculation purely because of the unit it happens to be measured in.

Why it is needed: on Titanic, `Age` runs 0.42 to 80 and `Fare` runs 0 to 512.33. Any algorithm that adds, weights or measures distance between features treats those two as if they were the same kind of quantity. In a Euclidean distance, a 10-pound difference in fare contributes 100 to the squared distance while a 10-year difference in age contributes the same 100, and the fare axis, being six times wider, ends up controlling every neighbourhood the algorithm computes. Scaling removes the unit from the comparison.

12.1 Which algorithms need it, measured

Five-fold cross-validated accuracy on one fixed Titanic feature set (`Pclass`, `Sex` as 0/1, `Age` median-imputed, `Fare`, `SibSp`, `Parch` and one-hot `Embarked`), with and without `StandardScaler` and changing nothing else:

Model	Unscaled	Scaled	Change	Why
SVC, RBF kernel	0.6767	**0.8282**	+15.2 pts	The kernel is a distance. `Fare` swamps every other feature.
kNN, $k = 5$	0.7048	**0.8126**	+10.8 pts	Neighbourhoods are defined by distance, so they are defined by `Fare`.
Logistic Regression	0.7924	0.7935	+0.1 pts	Only the optimiser cares. The solution is the same; scaling makes it converge faster and makes the coefficients comparable.
Decision Tree	0.7800	0.7845	noise	Splits are rank-based. The difference is tie-breaking randomness, not scaling.
Random Forest	0.8193	0.8182	noise	Same reason.

That table is the entire decision procedure. Anything that measures distance or sums weighted features needs scaling, sometimes desperately: a 15-point accuracy swing on an SVM is the difference between a working model and a broken one. Anything that splits on a threshold does not, and scaling it is harmless busywork.

Requires scaling	Does not require scaling
kNN, K-Means, DBSCAN, hierarchical clustering (distance)	Decision Tree
SVM with any kernel (distance)	Random Forest, Extra Trees
PCA, LDA, factor analysis (variance-based)	Gradient boosting: XGBoost, LightGBM, CatBoost
Neural networks (gradient stability)	Naive Bayes
Ridge, Lasso, ElasticNet (the penalty is applied to the coefficients, so it must see comparable scales)	Rule-based models
Gradient descent generally, for convergence speed	Plain linear and logistic regression, for correctness

The subtlety in that last row: plain logistic regression does not *need* scaling to be correct: multiplying a feature by 1000 divides its coefficient by 1000 and the predictions are unchanged. But regularised regression does, because the penalty $\lambda \sum w_j^2$ punishes large coefficients regardless of why they are large, and an unscaled feature with a small numeric range gets an unfairly large coefficient and is therefore penalised hardest. Since `LogisticRegression` in scikit-learn is regularised by default, in practice you scale it.

12.2 Standardization (Z-score)

Concept: Recentre the column on zero and rescale it so one unit equals one standard deviation. The transformed value answers a single question: how many standard deviations from the mean is this observation?

$$z = \frac{x - \mu}{\sigma}$$

Interpreting mean 0 and std 1: after the transform the column's mean is exactly 0 and its standard deviation exactly 1. A value of +2 means two standard deviations above average, and it means that identically whether the column was fares in pounds or ages in years. That is what makes coefficients comparable across features.

What it does NOT do: make the distribution normal. Standardizing `Fare` gives a mean of 0 and a standard deviation of 1, and leaves the skew at 4.79 and the maximum at +9.66 standard deviations. The shape is untouched; only the location and the unit change. If you need normality, transform first, with `log1p`, Box-Cox or a `QuantileTransformer`, and then scale.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train[["Age", "Fare"]] = scaler.fit_transform(X_train[["Age", "Fare"]])
X_test[["Age", "Fare"]]  = scaler.transform(X_test[["Age", "Fare"]])

scaler.mean_, scaler.scale_   # the learned parameters -- store these
```

Use cases: Logistic and linear regression, SVM, PCA, LDA, neural networks, and any situation where you want to compare feature importances by coefficient magnitude. It is the default. Choose it unless you have a reason not to.

12.3 Normalization (Min-Max scaling)

Concept: Linearly rescale the column so its minimum becomes 0 and its maximum becomes 1. Every value lands in $[0, 1]$ and the relative spacing between values is preserved exactly.

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

```
from sklearn.preprocessing import MinMaxScaler

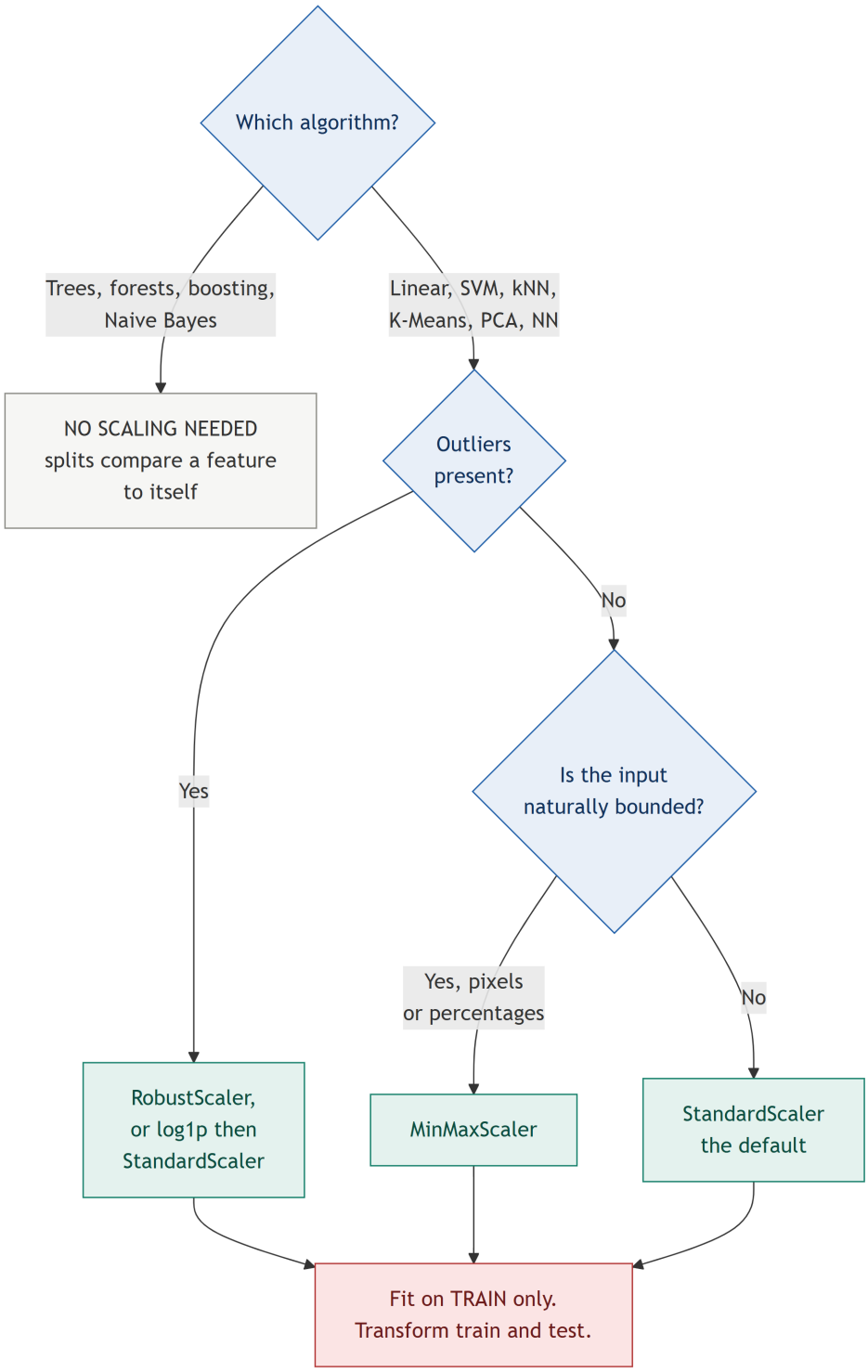
scaler = MinMaxScaler()                    # or feature_range=(-1, 1)
X_train[["Age", "Fare"]] = scaler.fit_transform(X_train[["Age", "Fare"]])
X_test[["Age", "Fare"]]  = scaler.transform(X_test[["Age", "Fare"]])
```

Use cases: Neural networks whose activations expect a bounded input; image data, where the natural bound is 0 to 255 and dividing by 255 is min-max scaling by another name; distance-based algorithms on features that are already bounded; and any algorithm that strictly requires non-negative input, such as non-negative matrix factorisation.

The catastrophic failure mode, on this dataset: min-max scaling `Fare` maps the three 512.33 tickets to 1.0 and crushes everything else into the bottom of the range. After the transform, the *median* passenger sits at **0.0282** and the 99th percentile at **0.4860**. Half the dataset is squeezed into the bottom 3% of the scale by three rows. Two outliers define the entire mapping, because two values, the minimum and the maximum, define the entire mapping.

12.4 Standardization versus normalization

	Standardization	**Normalization**
Formula	$(x - \mu)/\sigma$	$(x - x_{\min})/(x_{\max} - x_{\min})$
Output range	Unbounded, typically -3 to $+3$	Exactly $[0, 1]$
Centre	Mean at 0	Minimum at 0
Uses	Mean and standard deviation	Minimum and maximum
Outlier sensitivity	Moderate: outliers inflate σ, which shrinks everything else	**Severe**: two points define the whole mapping
Distribution assumption	Works best on roughly symmetric data; assumes nothing formally	None, but assumes the observed range is the real range
Unseen data outside the range	Handled naturally; a new value simply gets a larger $\lvert z \rvert$	Falls outside $[0, 1]$, breaking the guarantee the model relies on
Preserves shape	Yes	Yes
Typical algorithms	Linear, logistic, SVM, PCA, most neural nets	Image pixels, bounded-input nets, NMF
On Titanic `Fare`	Max at $+9.66\sigma$, median near -0.36	Median at 0.028, 99th percentile at 0.486



IF THE DATA HAS OUTLIERS: use `RobustScaler`, which centres on the median and divides by the IQR, or transform the column first with `log1p` and then standardize. Do not use min-max. On `Fare` the robust version puts the median at 0 and the 512.33 ticket at 21.56, which is a large number honestly reported rather than a distribution silently flattened.

IF THE DATA IS BOUNDED: min-max is a natural fit, because the theoretical minimum and maximum are known rather than estimated from the sample. Pixel intensities, percentages and normalised scores qualify.

IF YOU ARE USING PCA: standardize, always. PCA finds directions of maximum variance, and variance is measured in the feature's own units. Unscaled, `Fare` with a variance of 2469

dwarfs `Age` with a variance of 211, and the first principal component is simply the fare axis.

IF YOU ARE USING NEURAL NETWORKS: either works, and both are much better than neither. Standardization is the more common default for tabular data; min-max is standard where the input is bounded, such as images. What matters is that the inputs are small and centred, so the gradients are stable.

```
from sklearn.preprocessing import RobustScaler, PowerTransformer

RobustScaler()                              # (x - median) / IQR
PowerTransformer(method="yeo-johnson")      # makes it more Gaussian, then standardizes

# For Fare specifically, the two-step version is usually best:
X["Fare"] = np.log1p(X["Fare"])             # skew 4.79 -> 0.39
X["Fare"] = StandardScaler().fit_transform(X[["Fare"]])
```

Common mistakes

- **Fitting the scaler on the full dataset.** The mean and standard deviation are learned parameters. Fitting them on test rows is leakage, and it is the single most common leak in beginner pipelines.
- **Calling `fit_transform` on the test set.** It rescales the test set with its own statistics, so train and test end up in different spaces.
- **Scaling the target** in a classification problem. `Survived` is a label, not a feature.
- **Scaling one-hot columns.** Harmless but pointless; they are already 0 and 1. Worse, it makes them unreadable in the coefficient table.
- **Min-max on data with outliers.** 3% of the scale for half the dataset.
- **Scaling for a random forest** and believing it helped. It did not; the difference is tie-breaking noise.
- **Believing standardization creates normality.** It changes the location and the unit, never the shape.

Industry best practices

- **Scale inside a `Pipeline`.** Then `fit` on train and `transform` on test happen in the right order by construction, and cross-validation refits the scaler on every fold, which is the only correct behaviour.
- **Persist the fitted scaler** with the model. A model and a scaler fitted on different data are two incompatible artefacts.
- **Transform before scaling** when the column is skewed. `log1p` then `StandardScaler` beats either alone on `Fare`.
- **Use `ColumnTransformer`** so numeric columns are scaled and categorical ones are encoded in one object with one `fit`.
- **Monitor the scaled distribution in production.** If the incoming mean drifts away from 0, the input distribution has moved and the model is extrapolating.

Interview questions

1. **Standardization versus normalization: formulas and when to use each.**
   $(x - \mu)/\sigma$, *unbounded, the default and robust to a moderate tail;* $(x - x_{\min})/(x_{\max} - x_{\min})$, *bounded to* $[0, 1]$, *for genuinely bounded inputs and never with outliers.*

2. **Which algorithms need scaling and which do not?**
   Distance-based, gradient-based and variance-based methods need it: kNN, SVM, K-Means, PCA, neural nets, regularised regression. Tree-based methods and Naive Bayes do not.

3. **Why do trees not need scaling?**
   A split is a comparison of one feature against a threshold. Any monotonic transform of that feature leaves every possible split unchanged.

4. **Why must the scaler be fitted only on the training set?**
   Its mean and standard deviation are parameters learned from data. Learning them from test rows leaks information and inflates the reported score.

5. **Your data has extreme outliers. Which scaler?**
   `RobustScaler`, *or a log transform followed by* `StandardScaler`. *Not min-max.*

6. **Does standardization make the data normally distributed?**
   No. It sets the mean to 0 and the standard deviation to 1 and leaves the shape alone. `Fare` *keeps a skew of 4.79 either way.*

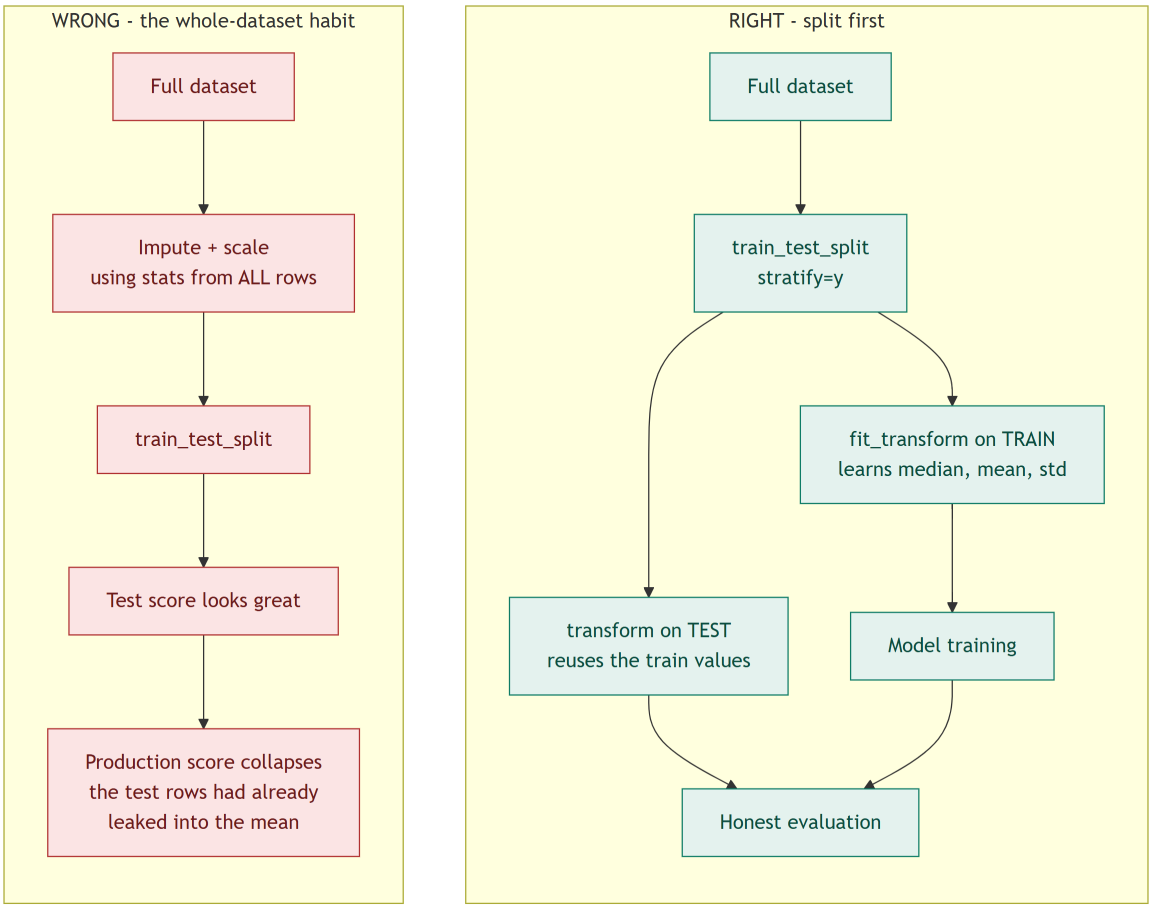
7. **Should you scale one-hot encoded columns?**
   Not necessary; they are already on a 0–1 scale. Doing it costs nothing but obscures interpretation.

13. Train / Test Split

Why splitting is required: a model's training error measures memory, not prediction. Any sufficiently flexible model can drive training error to zero by memorising the rows, and a decision tree grown to full depth on Titanic comes close: **98.17%** accuracy on the rows it was fitted to against **82.12%** on the held-out fifth. It misses a perfect training score only because some passengers share identical feature values and different outcomes. The only way to estimate how a model behaves on unseen data is to keep some data unseen.

13.1 Data leakage

Definition: Data leakage is any situation where information that would not be available at prediction time influences the model. Its signature is a test score that is too good and a production score that is not.



Type	What it looks like	Fix
Preprocessing leakage	Imputing, scaling or encoding before the split, so the statistics include test rows	Split first; `fit` on train, `transform` on both
Target leakage	A feature that could only be known after the outcome: a `lifeboat_number` column	Ask of every feature: was this knowable before the event?
Duplicate leakage	The same row in train and test	Deduplicate before splitting
Group leakage	Two members of one family split across train and test	`GroupShuffleSplit` on the group key
Temporal leakage	Training on the future to predict the past	`TimeSeriesSplit`; split by time, never at random
Tuning leakage	Choosing hyperparameters by test-set score, repeatedly	Use a validation set or cross-validation; touch the test set once

The Titanic group-leakage case: 681 unique tickets across 891 passengers, so 210 people travel with somebody. A random split puts the mother in train and her child in test. They share a fare, a cabin, a ticket and, very often, an outcome. The model is not generalising to a new passenger; it is looking up a family it has already seen. On this dataset the effect is small, but the pattern, groups split across the boundary, is the one that most often makes a model look better in testing than it is.

13.2 Train, validation and test

Set	Typical size	Purpose	How often you may look
Train	60–80%	Fit the model's parameters	Constantly
Validation	10–20%	Choose hyperparameters, compare models, decide when to stop	Many times; that is what it is for
Test	10–20%	Estimate performance on unseen data	**Once**, at the very end

Why the validation set exists: if you tune against the test set, you are fitting your choices to it and its score stops being an estimate of anything. The validation set absorbs that wear. In practice, with a dataset this small, cross-validation replaces a fixed validation set: it uses every row for both training and validation across folds, and gives you a standard deviation as well as a mean.

13.3 The code, and the three arguments

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["Survived", "PassengerId", "Name", "Ticket", "Cabin"])
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 179 test rows, 712 train rows
```

```
    random_state=42,        # reproducible
    stratify=y,             # preserve the 38.38% survival rate in both halves
)
```

Argument	What it controls
`test_size`	The held-out fraction. 0.2 is the usual default. Small datasets need a larger test set for a stable estimate but cannot afford one, which is what cross-validation resolves.
`random_state`	The seed for the shuffle. Fixing it makes the split reproducible; without it, every rerun gives a different score and no comparison between two models means anything.
`stratify`	Preserves the class proportions in both halves. Pass the target for classification.

What `stratify` actually buys, measured: across 20 different seeds, the survival rate in the test set ranges from **33.52% to 44.13%** without stratification. With `stratify=y` it is **38.55% every time**, against a population rate of 38.38%. A 10-point swing in the test set's class balance is a 10-point swing in what the score even means. Stratification costs one argument.

What `random_state` hides, also measured: the same logistic regression pipeline, evaluated on 20 different stratified 80/20 splits, scores between **0.7989 and 0.8771**. That is a spread of **7.8 accuracy points** from nothing but the choice of seed. Any comparison of two models on one split, where the difference is under a point, is measuring the seed. This is the argument for cross-validation, and it is not a theoretical one.

13.4 Cross-validation

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipeline, X, y, cv=cv, scoring="accuracy")
print(f"{scores.mean():.4f} +/- {scores.std():.4f}")
# Logistic regression: 0.8249 +/- 0.0133
# Random forest:       0.8215 +/- 0.0188
```

Report the standard deviation, always. 0.8249 alone is a number. 0.8249 ± 0.0133 is a result. The two models above differ by 0.3 points with standard deviations of 1.3 and 1.9 points, which is the honest way of saying they are not distinguishable on this dataset. Note also that the random forest is the less stable of the two, which is itself worth knowing.

Strategy	When
`KFold`	Regression, or classification with a balanced target
`StratifiedKFold`	Classification. The default choice; preserves class balance in every fold
`GroupKFold`	Rows belong to groups that must not straddle the split: families, patients, users
`TimeSeriesSplit`	Ordered data. Every fold trains on the past and tests on the future
`LeaveOneOut`	Very small datasets. Expensive and high-variance
Repeated stratified k-fold	When you need a tighter estimate and can afford the compute

The pipeline must go inside the cross-validation, not around it. Passing a fitted transformer to `cross_val_score` leaks every fold's validation rows into the transformer's statistics. Passing a `Pipeline` object refits the imputer, encoder and scaler on each training fold, which is the only correct behaviour and requires no discipline from you.

Common mistakes

- **Preprocessing before splitting.** The default beginner pipeline, and the default interview question.
- **Not stratifying an imbalanced target.** 10 points of class-balance swing.
- **No `random_state`.** Results that change on every run and cannot be compared.
- **Tuning on the test set.** After the tenth look it is a training set.
- **Random splitting of time series.** Training on the future to predict the past.
- **Ignoring groups.** Families, sessions and patients must stay on one side.
- **Reporting a single split's score** as though it were precise. On this data the seed alone is worth 7.8 points.

Industry best practices

- **Split first, before anything else touches the data.** Make it the first line after loading.
- **Put every learned transformation in a `Pipeline`**, so leakage is structurally impossible rather than remembered.
- **Cross-validate for model selection, hold out for the final estimate.**
- **Split by time when the model will predict the future**, even if the data is not obviously a time series.
- **Keep a genuinely untouched test set** for the final report, and record how many times it has been evaluated. The honest answer is one.
- **Report mean and standard deviation.** A single number invites a conclusion it cannot support.

Interview questions

1. **What is data leakage and how do you prevent it?**
   Information available in training that would not be available at prediction time. Prevent it by splitting first and fitting every transformer inside a pipeline on the training fold only.
2. **Why do you need a validation set when you already have a test set?**
   Because choosing hyperparameters against the test set fits your decisions to it, and its score stops being an unbiased estimate.
3. **What does `stratify` do and when does it matter?**
   Preserves class proportions in both halves. It matters whenever the target is imbalanced or the dataset is small; here it removes a 10-point swing in the test set's positive rate.
4. **Why fix `random_state`?**
   Reproducibility. Without it, two runs of the same code differ, and on this dataset the seed alone moves accuracy by up to 7.8 points.

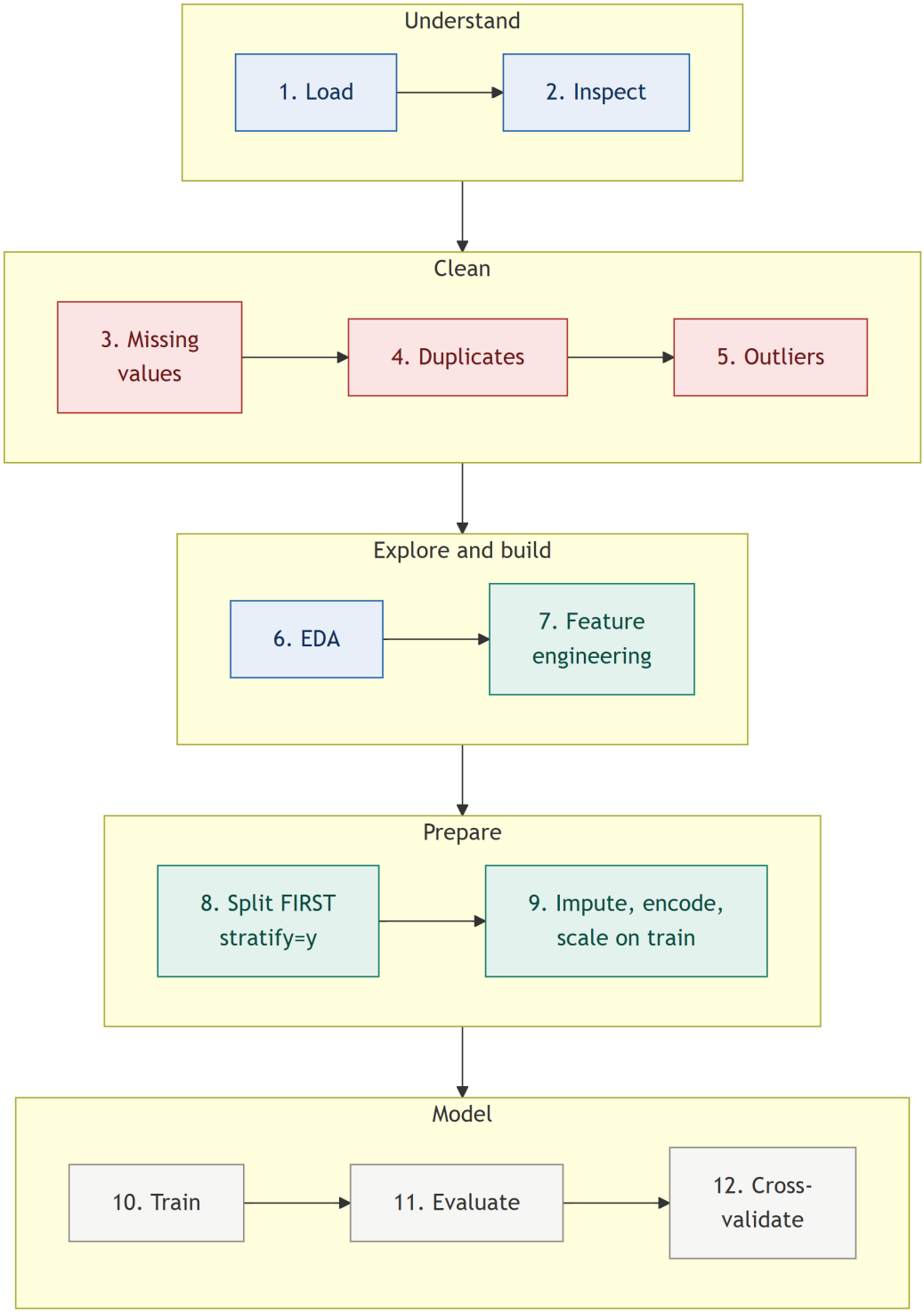
5. **How do you split time-series data?**
   By time. Train on the past, test on the future, with `TimeSeriesSplit`*. A random split leaks the future.*

6. **You have 500 rows. 80/20 split or cross-validation?**
   Cross-validation. A 100-row test set gives an estimate too noisy to act on, and k-fold uses every row for both purposes.

7. **Your test accuracy is much higher than your production accuracy. First hypothesis?**
   Leakage: preprocessing fitted on the full dataset, duplicate rows across the split, a target-derived feature, or groups straddling the boundary.

14. Complete Titanic Pipeline

Everything in the preceding thirteen sections, as one script that runs top to bottom. The order differs from the teaching diagram in Section 1 in exactly one respect, and it is the important one: **the split happens before anything is fitted**.



14.1 Steps 1 and 2: load and understand

```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report)
from sklearn.model_selection import (train_test_split, cross_val_score,
                                     StratifiedKFold)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 42

raw = pd.read_csv("Titanic.csv")
df  = raw.copy()
assert df.shape == (891, 12), f"unexpected shape {df.shape}"

df.info()
df.describe()
df.describe(include="object")
df.isnull().sum()[lambda s: s > 0]     # Age 177, Cabin 687, Embarked 2
df["Survived"].value_counts(normalize=True)   # 0.6162 / 0.3838
```

14.2 Steps 3 to 5: cleaning decisions, recorded

```
# 4. Duplicates: none, but check with the surrogate key removed
assert df.drop(columns=["PassengerId"]).duplicated().sum() == 0

# 5. Outliers: inspected, not removed. Fare is right-skewed (4.79) with 116
#    rows beyond the 1.5*IQR fence, and those rows survive at 68.1% against a
#    38.4% baseline. Treated by transformation (log1p) inside the pipeline,
#    never by deletion.
Q1, Q3 = df["Fare"].quantile([0.25, 0.75])
upper  = Q3 + 1.5 * (Q3 - Q1)          # 65.6344
```

14.3 Step 7: feature engineering, row-wise and leak-free

```
def engineer(data: pd.DataFrame) -> pd.DataFrame:
    """Every feature here is computed from one row, or from a mapping that is
    fitted later inside the pipeline. Nothing uses a full-column statistic."""
    out = data.copy()

    out["FamilySize"] = out["SibSp"] + out["Parch"] + 1
    out["IsAlone"]    = (out["FamilySize"] == 1).astype(int)
    out["HasCabin"]   = out["Cabin"].notna().astype(int)
    out["Deck"]       = out["Cabin"].str[0].fillna("Unknown")

    title = out["Name"].str.extract(r",\s*([^\.]+)\.")[0].str.strip()
    title = title.replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"})
    out["Title"] = title.where(
        title.isin(["Mr", "Mrs", "Miss", "Master"]), "Rare"
    )

    out["LogFare"] = np.log1p(out["Fare"])      # skew 4.79 -> 0.39
```

```
    return out


df = engineer(df)

FEATURES = ["Pclass", "Sex", "Age", "LogFare", "Embarked",
            "Title", "FamilySize", "IsAlone", "HasCabin"]

X = df[FEATURES]
y = df["Survived"]
```

14.4 Step 8: split, before anything is fitted

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y,
)
print(len(X_train), len(X_test))              # 712, 179
print(y_train.mean(), y_test.mean())          # 0.3834, 0.3855
```

14.5 Step 9: one transformer for the whole matrix

```
numeric     = ["Age", "LogFare", "FamilySize"]
categorical = ["Pclass", "Sex", "Embarked", "Title"]
passthrough = ["IsAlone", "HasCabin"]         # already 0/1

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale",  StandardScaler()),
    ]), numeric),

    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(drop="first", handle_unknown="ignore")),
    ]), categorical),

    ("pass", "passthrough", passthrough),
])
```

Why this object is the whole point of the section: it holds every learned parameter, the medians, the modal categories, the means and standard deviations, the category lists, in one place, and it exposes exactly one `fit`. Called inside a pipeline, it is fitted on the training fold and applied to the test fold. The leak from Section 13 is not avoided by discipline; it is unavailable.

14.6 Steps 10 to 12: train, evaluate, cross-validate

```
models = {
    "LogisticRegression": LogisticRegression(max_iter=1000,
                                             random_state=RANDOM_STATE),
    "RandomForest": RandomForestClassifier(n_estimators=300,
                                           random_state=RANDOM_STATE),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

for name, estimator in models.items():
```

```
    pipe = Pipeline([("prep", preprocess), ("model", estimator)])
    pipe.fit(X_train, y_train)

    pred  = pipe.predict(X_test)
    proba = pipe.predict_proba(X_test)[:, 1]

    print(name)
    print(f"  accuracy  {accuracy_score(y_test, pred):.4f}")
    print(f"  precision {precision_score(y_test, pred):.4f}")
    print(f"  recall    {recall_score(y_test, pred):.4f}")
    print(f"  f1        {f1_score(y_test, pred):.4f}")
    print(f"  roc_auc   {roc_auc_score(y_test, proba):.4f}")
    print(" ", confusion_matrix(y_test, pred).tolist())

    scores = cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")
    print(f"  5-fold CV {scores.mean():.4f} +/- {scores.std():.4f}")
```

14.7 The results

Model	Acc	Prec	Recall	F1	ROC-AUC	5-fold CV
Logistic Regression	0.8101	0.7778	0.7101	0.7424	0.8596	0.8249 ± 0.0133
Random Forest	0.7877	0.7313	0.7101	0.7206	0.8351	0.8215 ± 0.0188
Predict “died” always	0.6162	—	0.0000	—	0.500	—
Predict “female survives”	0.7868	—	—	—	—	—

Read the baselines before the models. Predicting the majority class scores 61.62%. A one-line rule, *women survive, men do not*, scores **78.68%**. The logistic regression’s 81.01% is therefore worth about **2.3 points** over a rule you could write on a napkin. That is a real gain and a small one, and reporting it without the baseline would be misleading. Every model report should carry the number it has to beat.

Reading the confusion matrix: the logistic regression’s test predictions are $[[96, 14], [20, 49]]$: 96 correctly predicted deaths, 14 passengers predicted to survive who did not, 20 predicted to die who survived, and 49 correctly predicted survivors. Recall of 0.7101 means the model misses 29% of actual survivors. Whether that matters depends entirely on what the prediction is for, which is a question the accuracy number cannot answer.

Why the simpler model wins here: 891 rows, nine features, and a strong linear signal in `Sex` and `Pclass`. The random forest has more capacity than the data can support and its cross-validated standard deviation is 40% larger. On this dataset the extra flexibility buys variance rather than accuracy.

14.8 Production checklist

Stage	Check before moving on
Load	Shape asserted. Source, version and pull date recorded. Raw copy retained.
Understand	`info`, `describe` and `describe(include="object")` read, not just run. Column types classified by meaning.
Missing	Rate per column measured. Mechanism named. Decision and reason written down. Indicator columns added where the rate is high.
Duplicates	Checked on a business key, not on all columns. Count logged.
Outliers	Detected, *inspected as rows*, classified as error or valid. Treatment justified. Bounds fitted on train.
EDA	Univariate for every column, bivariate against the target, multivariate for confounders. One written conclusion per finding.
Features	Each one row-wise or fitted inside the pipeline. Each one validated by a cross-validated score delta.
Split	**Before any fitting.** Stratified. Seeded. Groups respected if they exist.
Preprocess	Inside a `ColumnTransformer` inside a `Pipeline`. Fitted on train only.
Train	Baseline computed first. Seed fixed.
Evaluate	Metric matches the problem, not the habit. Confusion matrix inspected. Compared against the baseline.
Cross-validate	Mean and standard deviation reported. The whole pipeline inside the folds.
Ship	Pipeline, feature code and metrics persisted together as one versioned artefact.

15. Industry Best Practices

15.1 What changes when the work goes to production

A notebook has one job: produce a number that convinces someone. A production system has a harder one: produce the same number every day, on data nobody has looked at, and fail loudly when it cannot.

	In a notebook	**In production**
Preprocessing	Cells run in whatever order you last ran them	One `Pipeline` object, fitted once, serialised, versioned
Missing values	Filled ad hoc	Imputer fitted on train, values stored, missing rate monitored
Features	Written inline	One function per feature, shared verbatim by training and serving
Splitting	`train_test_split`	Split by time or by group, matching how predictions will actually be made
Evaluation	Accuracy	The metric the decision needs, against an explicit baseline, with a confidence interval
The data	A fixed CSV	A moving distribution with a schema contract and drift alerts
Reproducibility	"It worked yesterday"	Pinned seeds, pinned dependencies, pinned data version
Failure	The cell errors	The model returns a confident wrong answer and nobody notices for a month

The five habits that separate the two:

1. **Everything learned lives in one fitted object.** If a median, a category list, a mean, a standard deviation or a bin edge exists anywhere outside the pipeline, it will eventually be recomputed differently in serving. This single discipline prevents most production incidents in tabular ML.
2. **The baseline is computed first.** Majority class, then a one-rule model. On Titanic those are 61.62% and 78.68%. A model that cannot beat both is not a model.
3. **Every number is reported with its uncertainty.** 0.8249 ± 0.0133, not 0.8249. The second form invites conclusions the data cannot support.
4. **The test set is touched once.** Cross-validate to choose; hold out to report.
5. **The data is monitored, not just the model.** Missing rates, category frequencies, and the mean of each scaled feature. Almost every "the model degraded" incident is really "the input changed".

15.2 Common beginner mistakes, collected

Stage	Mistake	Correct approach
Loading	Trusting inferred dtypes and skipping `tail()`	Assert the shape, set dtypes explicitly, read both ends
Types	Averaging or scaling an ordinal or an ID	Classify by meaning, not by dtype
Missing	Blanket `dropna()`	Measure the cost first: on Titanic it is 891 rows down to 183
Missing	Mean-imputing a skewed column	Median, and check the skew before choosing
Missing	Dropping a 77%-missing column outright	Extract the signal first: `HasCabin` is worth 37 points
Duplicates	Checking with the ID column present	Drop identifiers, or use an explicit business key
Outliers	Deleting everything beyond 1.5 IQR	Inspect the rows; transform or cap; delete only confirmed errors
EDA	Reading counts as rates	Convert to a rate and always report n
EDA	Dropping features on low Pearson r	Check for non-linearity; `Age` at -0.077 is useful once binned
Features	Using full-dataset statistics	Row-wise features, or fit inside the pipeline
Encoding	Label-encoding a nominal for a linear model	One-hot, with `drop_first`
Encoding	`get_dummies` separately on train and test	`OneHotEncoder` with `handle_unknown`
Scaling	`fit_transform` on the test set	`fit` on train, `transform` on both
Scaling	Min-max on a column with outliers	`RobustScaler`, or log then standardize
Splitting	Preprocessing before splitting	Split first, always
Splitting	Reporting one split's score as precise	Cross-validate; the seed alone is worth 7.8 points here
Evaluation	Accuracy on imbalanced data	Precision, recall, F1, ROC-AUC, and the confusion matrix
Evaluation	No baseline	Majority class and a one-rule model, computed first

15.3 Checklist: before training any model

#	Check
01	Shape asserted; source, version and pull date recorded.
02	Every column classified by meaning, not by dtype; identifiers excluded from the feature list.
03	Missing rate measured per column; mechanism named; decision and reason written down.
04	Duplicates checked on a business key; the count logged.
05	Outliers detected, inspected as rows, and classified as error or valid before any treatment.
06	Target distribution known; the class balance drives the metric and the split.
07	Univariate view of every column; bivariate view against the target; confounders checked.
08	Every feature answerable to the question: could this be computed for one new row, alone?
09	Train/test split performed *before* any transformer is fitted; stratified and seeded.
10	Every learned transformation inside a `Pipeline` or `ColumnTransformer`.
11	Baseline scores computed: majority class, and a single-rule model.
12	Metric chosen to match the decision the model supports, not out of habit.
13	Random seeds fixed everywhere: split, model, cross-validation.
14	No column in the feature set could only be known after the outcome.
15	The whole notebook runs top to bottom on a fresh kernel.

15.4 Checklist: before deploying any model

#	Check
01	Cross-validated score reported with its standard deviation, and it beats both baselines.
02	The test set has been evaluated exactly once, and the number is recorded.
03	Pipeline, feature code, metrics and data version serialised as one artefact.
04	Training and serving share the same feature code, not two implementations of it.
05	Dependencies pinned; the artefact reloads and predicts in a clean environment.
06	Unseen categories, missing columns and out-of-range values handled explicitly, not by exception.
07	Prediction latency and payload size measured against the requirement.
08	Input monitoring live: missing rates, category frequencies, feature means.
09	Output monitoring live: prediction distribution and, when labels arrive, realised performance.
10	Alert thresholds set, with a named owner for each.
11	Rollback path tested, not just documented.
12	Errors examined by segment; a model that is accurate overall and wrong for one group is not ready.
13	Model card written: training data, assumptions, metrics, known limitations, intended use.
14	Retraining trigger defined: on a schedule, on drift, or on a performance floor.
15	The decision the model supports is written down, along with what happens when it is wrong.

Appendix: The Guide on One Page

Question	Answer	Titanic
Column is 80% missing?	Extract the signal, then drop	`Cabin` → `HasCabin`, `Deck`
Column is 20% missing, numeric?	Median, grouped if MAR	`Age` by `Title` × `Pclass`
Column is 0.2% missing, categorical?	Mode	`Embarked` → `S`
Mean ≫ median?	Right skew: median, and log-transform	`Fare`: 32.20 vs 14.45
|skew| < 0.5?	Mean imputation is safe	`Age`: 0.39
|skew| > 1?	Median, and transform	`Fare`: 4.79 → 0.39 with `log1p`
Outlier found?	Error or valid? Then transform, cap, or leave	`Fare` 512.33: valid, a shared ticket
Model is a tree?	Skip scaling and outlier treatment	RF: 0.8193 unscaled, 0.8182 scaled
Model uses distance?	Scale, always	SVC: 0.6767 → 0.8282
Feature has outliers?	`RobustScaler`, or log then standardize	Never min-max on `Fare`
Category is binary?	Map to 0/1	`Sex`
Category is nominal, 3–15 levels?	One-hot, `drop_first`	`Embarked`, `Title`
Category is ordinal?	Keep the order	`Pclass` as 1, 2, 3
Category has 600 levels?	Do not one-hot; extract a feature	`Name` → `Title`
Two columns each useless?	Try combining them	`SibSp` + `Parch` → `FamilySize`
Relationship is non-linear?	Bin it, for a linear model	`Age`: 57.97% under 12, 22.73% over 60
About to fit anything?	Split first	`stratify=y`, seeded
Target is imbalanced?	Stratify; report more than accuracy	38.38% survived
Reporting a score?	With a standard deviation and a baseline	0.8249 ± 0.0133 vs 0.7868

The one sentence to keep: every number a transformer learns, a median, a mode, a mean, a standard deviation, a category list, a bin edge, is a model parameter, and it must be learned from the training data alone. Everything else in this guide is a consequence of that.